

الجامعة الإسلامية بغزة

The Islamic University of Gaza

كلية تكنولوجيا المعلومات

Faculty of Information Technology

Smart Management System for Mobile Charging Shops in the Gaza Strip

نظام إدارة ذكي لمحلات شحن الأجهزة المحمولة في قطاع غزة

By

Mohammed Eyad Abo Madkur — 120197227

Mohammed Hisham Mahdi — 120213237

Hussein Ezzat Ghabayen — 120211234

Mahmoud Mohammed Mady — 120212580

Supervised by

Dr. Raed M. I. Bulbul

**A graduation project report submitted in partial fulfillment of the requirements for the
degree of Bachelor of Information Technology**

August 2026

Abstract

The continuous electricity crisis in the Gaza Strip has turned mobile-device charging shops into an essential daily service relied upon by individuals, students, and small-business owners. These shops are still managed manually, using paper notebooks and the owner's memory, which leads to forgotten debts, calculation errors, difficulty tracking devices across numbered shelves, and the absence of any reports to support decision-making.

This project presents a smart management system, delivered as an Android mobile application with a fully Arabic (right-to-left) interface built with Flutter. The application communicates over HTTPS with a RESTful backend developed using Node.js and Express, backed by a PostgreSQL relational database, and secured through Firebase Authentication. The backend and database are deployed on the Render cloud platform, so the application runs against a live production environment. The system covers the complete work cycle: receiving devices onto numbered shelves with automatic pricing from a unified price list, direct retail sales of complementary goods, smart delivery with on-the-spot payment collection, per-customer account statements, a debtors list, and a full operations log with daily, weekly, and monthly filters alongside a live dashboard.

The core of the system is a unified balance accounting model in which the customer's balance is the single source of truth: every transaction deducts its value from the balance and every deposit adds to it, so the payment status is derived automatically rather than entered manually, while the balance update logic and ACID transactions guarantee financial consistency. The system was developed following a sequential Waterfall methodology and field-tested inside a real, operating charging shop, where the accuracy of the accounts was verified by reconciling each customer's balance against the sum of their transactions and deposits.

The results show that the system reliably automates daily operations, keeps debts and balances clear and traceable, and provides instant indicators to the shop owner, with a measured median API response time of approximately 552 ms once the cloud service is warm. In conclusion, the project demonstrates a practical, generalizable model of digital transformation for small businesses operating under difficult infrastructure conditions, with a multi-tenant design that paves the way for offering it as a service to similar shops.

ملخص الدراسة

أدت أزمة الكهرباء المستمرة في قطاع غزة إلى تحوّل محلات شحن الأجهزة المحمولة إلى خدمة يومية أساسية يعتمد عليها الأفراد والطلبة وأصحاب الأعمال الصغيرة. وما تزال هذه المحلات تُدار يدوياً بالاعتماد على الدفاتر الورقية وذاكرة صاحب المحل، مما يؤدي إلى ضياع الديون، وأخطاء الحساب، وصعوبة تتبّع الأجهزة على الرفوف المرقّمة، وغياب أي تقارير تساعد على اتخاذ القرار.

يقدم هذا المشروع نظام إدارة ذكياً على هيئة تطبيق أندرويد بواجهة عربية كاملة، مبنياً بتقنية Flutter، يتواصل عبر HTTPS مع خدمات خلفية RESTful مطوّرة بـ Node.js و Express، تعتمد على قاعدة بيانات علائقية PostgreSQL، ومؤمنة بخدمة Firebase Authentication. والخادم وقاعدة البيانات منشوران سحابياً على منصة Render، فيعمل التطبيق على بيئة إنتاج حقيقية. ويغطي النظام دورة العمل كاملة: استلام الأجهزة على رفوف مرقّمة بأسعار تلقائية من قائمة أسعار موحّدة، والبيع المباشر للسلع المرافقة، والتسليم الذكي مع تحصيل الدفعات، وكشوف حساب لكل عميل، وقائمة بالمدينين، وسجل عمليات شامل بفلتر يومية وأسبوعية وشهرية إلى جانب لوحة مؤشرات لحظية.

يقوم جوهر النظام على نموذج محاسبي موحّد يكون فيه رصيد العميل هو المصدر الوحيد للحقيقة: كل عملية تخصم قيمتها من الرصيد وكل إيداع يضيف إليه، فتُشتق حالة الدفع تلقائياً دون إدخال يدوي، بينما يضمن منطق تحديث الرصيد ومعاملات ACID اتساق البيانات المالية. طُوّر النظام وفق منهجية Waterfall المتسلسلة واختُبر ميدانياً داخل محل شحن حقيقي قيد التشغيل، حيث جرى التحقق من دقة الحسابات بمطابقة رصيد كل عميل مع مجموع عملياته وإيداعاته.

أظهرت النتائج أن النظام يؤتمت العمليات اليومية بموثوقية، ويُبقي الديون والأرصدة واضحة وقابلة للتتبع، ويوفّر مؤشرات فورية لصاحب المحل، بزمان استجابة وسيط للخدمات بلغ نحو 552 ملي ثانية عندما تكون الخدمة السحابية نشطة. وخلص المشروع إلى تقديم نموذج عملي قابل للتعميم للتحويل الرقمي في المشاريع الصغيرة العاملة في ظروف بنية تحتية صعبة، بتصميم متعدد المستأجرين يمهد لتقديمه كخدمة لمحلات مشابهة.

Dedication

To our parents, whose patience, prayers, and enduring support carried us through every
hardship;

To the people of Gaza, whose resilience in the face of daily challenges inspired the very
problem this project seeks to solve;

And to every small-business owner who keeps working despite the difficult circumstances —
we dedicate this work.

Acknowledgment

First and foremost, we thank Allah Almighty for granting us the strength, patience, and guidance to complete this project.

We extend our sincere gratitude to our supervisor, Dr. Raed M. I. Bulbul, for his valuable guidance, continuous feedback, and encouragement throughout the different stages of this work.

We are also grateful to the Faculty of Information Technology at the Islamic University of Gaza for the knowledge and academic foundation it provided us during our study.

Finally, we thank the owner of the charging shop who allowed us to deploy and field-test the system on real daily operations, whose practical feedback greatly improved the final product.

Table of Contents

Abstract	2
ملخص الدراسة.....	3
Dedication	4
Acknowledgment	5
Table of Contents	6
List of Tables	9
List of Figures	10
List of Abbreviations	11
Chapter 1 __ Introduction	12
1.1 Problem Statement	12
1.2 Objectives	12
1.2.1 Main Objective	12
1.2.2 Sub Objectives	12
1.2.3 Beneficiaries	13
1.3 Scope and Limitations	13
1.3.1 Assumptions	13
1.4 Importance of the Project	13
1.4.1 Project Contributions	14
1.5 Report Roadmap	14
Chapter 2 __ Background and Related Work	15
2.1 International Point-of-Sale (POS) Platforms	15
2.1.1 Square Point of Sale	15
2.1.2 Loyverse POS	16
2.1.3 Feature-Level Comparison of the Two POS Platforms	16
2.2 Global Digital Debt-Ledger Applications	16
2.3 Arabic-Market Accounting Platforms	17
2.3.1 Daftra	17
2.3.2 Qayd	18
2.3.3 Feature-Level Comparison of the Two Arabic-Market Platforms	18
2.4 Comparative Analysis	19
2.5 Why None of the Reviewed Products Was Adopted	19
2.6 The Gap This Project Fills	20
2.7 Theoretical Background	20
Chapter 3 __ Methodology and Project Management	22
3.1 Methodology Name and Rationale	22

3.2 Key Phases of the Methodology	22
3.3 Adaptations to the Standard Methodology	22
3.4 Tools and Equipment	23
3.5 Timetable	23
3.6 Team Management	24
3.6.1 Team Structure and Roles	24
3.6.2 Communication and Collaboration	25
3.6.3 Conflict Resolution and Delegation	25
3.7 Risks and Changes	25
Chapter 4__ Requirements Analysis and System Design	26
4.0 Requirements Elicitation	26
4.1 System Architecture	26
4.2 Functional Requirements	27
4.3 Non-functional Requirements	28
4.4 Use-case Diagram	28
4.5 Database Design	30
4.6 The Unified Balance Accounting Model	31
4.7 Sequence Diagrams	32
Chapter 5__ Implementation	34
5.1 Backend Organization	34
5.2 Authentication and Owner Data Isolation	34
5.3 Payment and Debt Processing	34
5.4 REST API Endpoints	35
5.5 Frontend Organization	35
5.6 API Communication Layer	36
5.7 Application Screenshots	36
5.7.1 Home Screen and Dashboard Indicators	36
5.7.2 Registering a Charging Device on a Shelf	38
5.7.3 Quick Sale of a Retail Item	40
5.7.4 Smart Delivery with Payment Collection	41
5.7.5 Customer Statement	42
5.7.6 Debtors List	43
5.7.7 Filtered Operations Log	44
5.8 Deployment	45
Chapter 6__ Testing and Evaluation	46
6.1 Testing Strategy	46
6.2 Test Case Specification	46

6.3 User Acceptance Testing (UAT)	47
6.4 Performance Measurement (Revised)	47
6.5 Postman Collection	48
6.6 Evaluation Against Objectives	48
6.7 Concurrency Behavior	49
Chapter 7__Deployment, Security, and Limitations	50
7.1 Deployment Architecture	50
7.2 Cost	50
7.3 Backup Policy	50
7.4 Secrets Management	50
7.5 Transport Security and Authentication Lifetime	51
7.6 Fault Handling	51
7.7 Current Limitations	51
7.8 Customer Data Protection	52
Chapter 8__Conclusions and Future Work	54
8.1 Conclusions	54
8.2 Future Work	54
References	55
Appendix A: Repository Links and Design Resources	56
Appendix B: Selected Source Code	57
Appendix C: User Manual (Brief)	60

List of Tables

1	Table (1.1): Manual management vs. the proposed system.	14
2	Table (2.2): Feature-level comparison of Square and Loyverse.	16
3	Table (2.3): Feature-level comparison of Daftra and Qayd.	18
4	Table (2.4): Comparison between the proposed system and existing market solutions.	19
5	Table (3.1): Planned project timetable.	23
6	Table (3.2): Actual project timetable, mapped to planned weeks, with deviation reasons. ..	24
7	Table (4.1): Functional requirements, acceptance criteria, and traceability to testing.	28
8	Table (4.2): Non-functional requirements summary.	28
9	Table (5.1): REST API endpoints of the system.	35
10	Table (6.1): Test case specification, traced to functional and non-functional requirements.	47
11	Table (6.2a)Field usage by shop staff.....	47
12	Table (6.2b): UAT — internal team members.	47
13	Table (6.3): API response time across connectivity states (Postman; sign-in and GET /api/customers endpoints; Aug. 2026; broadband connection; 24 total samples).	48
14	Table (6.4): Evaluation of each measurable objective against its specific numeric criterion.	49

List of Figures

1	Figure (4.1): System architecture (three-tier)	27
2	Figure (4.2): Use-case diagram.	29
3	Figure (4.3): Entity-Relationship Diagram (ERD).	31
4	Figure (4.4): Sequence diagram — adding a device with a partial payment.	32
5	Figure (4.5): Sequence diagram — settling a customer debt.	33
6	Figure (5.1): Home screen and live dashboard	38
7	Figure (5.2): Registering a charging device on a shelf	39
8	Figure (5.3): Quick sale of a retail item	40
9	Figure (5.4): Smart delivery with payment collection	41
10	Figure (5.5): Customer statement	42
11	Figure (5.6): Debtors list	43
12	Figure (5.7): <i>Filtered operations log</i>	44

List of Abbreviations

Abbreviation	Meaning
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DB	Database
ERD	Entity-Relationship Diagram
HTTP(S)	Hypertext Transfer Protocol (Secure)
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
REST	Representational State Transfer
RTL	Right-to-Left
SQL	Structured Query Language
UAT	User Acceptance Testing
UI/UX	User Interface / User Experience

Chapter 1__Introduction

The Gaza Strip suffers from a chronic electricity crisis, with power available for only a few hours per day. In response, small shops that offer mobile-device charging as a paid service have spread widely and become part of daily life for many residents. Beyond charging phones, power banks, and batteries, these shops typically also sell complementary retail goods and maintain ongoing credit relationships with a base of regular customers.

A single shop may handle dozens of devices placed on numbered shelves during the day, while simultaneously selling items and tracking who owes money and who has a credit balance. In practice, all of this is managed manually through paper notebooks and the owner's memory. This manual approach is error-prone: debts are forgotten, arithmetic mistakes accumulate, the live status of shelves and devices is unclear, and the owner has no reports to understand the shop's performance. This project addresses that gap by delivering a smart, Arabic, mobile-first management system tailored to how these shops actually operate.

1.1 Problem Statement

Mobile-charging shops in Gaza are managed using manual, paper-based methods that do not scale with the volume of daily operations. This creates four concrete problems: (1) reliance on memory and notebooks causes forgotten debts and lost money with no trustworthy reference during disputes; (2) manual calculation of dozens of daily transactions leads to accumulating financial errors; (3) there is no live tracking of which device sits on which shelf, for whom, and whether it has been paid for; and (4) the absence of reporting makes it impossible to know the day's income, the outstanding debt, or the shop's overall performance without an exhausting manual audit.

1.2 Objectives

1.2.1 Main Objective

To design and implement a smart management system, delivered as an Arabic mobile application backed by cloud services, that automates the full operational and financial cycle of a mobile-charging shop and provides its owner with accurate, real-time information.

1.2.2 Sub Objectives

- O1: Automate registration of charging and sale transactions with automatic pricing (zero manual price entries required).

- O2: Implement a unified balance model in which 100% of payment statuses are derived automatically, with no manually-entered status fields.
- O3: Provide live shelf/device tracking, with state changes reflected within a single API call.
- O4: Generate customer statements, debtor lists, and filtered reports accessible in three taps or fewer from the home screen.
- O5: Enforce complete data isolation between shop owners, verified through dedicated test cases (see Chapter 6).

1.2.3 Beneficiaries

The system primarily benefits mobile-charging shop owners, who gain accurate financial tracking and real-time operational visibility. Indirectly, it benefits their regular customers, whose debts and credit balances are recorded reliably and transparently, reducing disputes. More broadly, it demonstrates a replicable model for other small, informal businesses in Gaza seeking affordable digital management tools.

1.3 Scope and Limitations

The project delivers an Android application with a fully Arabic interface, a RESTful backend, a cloud PostgreSQL database, authentication, the unified accounting logic, reporting, and multi-tenant data isolation. The following are intentionally out of the current scope and are discussed as future work: publishing to app stores, integrated online payment gateways, an iOS build, integration with external hardware (IoT), a dedicated customer-facing application, and an offline-first mode with local caching and background synchronization.

1.3.1 Assumptions

The design assumes that the shop owner's device has intermittent but sufficient internet access to reach the cloud-hosted backend, that a single owner manages one shop's data at a time, and that customers do not directly interact with the system — all operations are entered by the shop owner on their behalf.

1.4 Importance of the Project

The system is built and validated inside a real, operating shop rather than as a laboratory prototype, so every feature is exercised against genuine daily operations and real customer data. This grounds the project in a real problem and demonstrates a practical model of digital transformation for small businesses in a challenging environment. Its multi-tenant architecture also makes it generalizable: the same system could be offered as a service to the

many similar shops in the local market. Academically, it is a complete full-stack project spanning a mobile client, RESTful services, a cloud relational database, authentication, and deployment.

1.4.1 Project Contributions

This project contributes: (1) a unified, single-source-of-truth balance accounting model tailored to businesses combining services and retail sales; (2) a shelf-aware data model that represents physical charging slots as first-class entities; (3) a multi-tenant architecture enabling one system to serve many independent shops; and (4) a field-validated implementation, tested against real operations in an operating shop rather than synthetic data.

Table (1.1) contrasts the current manual management of a charging shop with the proposed system, summarizing the practical value the system adds across the main operational aspects.

Aspect	Manual Management	Proposed System
Debt & balance tracking	Notebooks and memory — prone to loss and error	One automatic, accurate balance per customer
Accounting accuracy	Manual addition/subtraction, accumulating errors	Computed automatically via the balance update logic and ACID transactions
Device & shelf status	Unclear, tracked mentally	Live on screen for every shelf and device
Customer statement	Not available	Unified statement (transactions + deposits) in one tap
Reporting	None — requires manual stock-taking	Instant: daily/weekly/monthly plus a live dashboard
Backup & recovery	Paper, easily damaged or lost	Stored securely in the cloud

Table (1.1): Manual management vs. the proposed system.

1.5 Report Roadmap

The remainder of this report is organized as follows. Chapter 2 reviews the market for existing solutions and positions the project against them. Chapter 3 describes the Waterfall methodology followed. Chapter 4 presents the requirements analysis and system design, including the database schema and the unified balance model. Chapter 5 details the implementation of the backend and mobile application. Chapter 6 reports the testing strategy and results. Chapter 7 covers deployment, security, and current limitations. Chapter 8 concludes the report and outlines future work.

Chapter 2__Background and Related Work

Before designing this system, the market was surveyed for existing software and methods already used, or available for use, by small shops to manage sales, inventory, and customer debt — both what commercial software already offers, and what the shop itself had already tried before this project began (Section 4.0 details the shop’s own history with paper, general sales apps, and spreadsheets). This chapter complements that first-hand history with a market review of five publicly available products — spanning international point-of-sale platforms, a global digital debt-ledger application, and Arabic-market accounting and debt-tracking platforms — describing for each how it actually works, what a shop owner sees and does when using it, and specifically why it falls short of this project’s needs. The chapter closes with a structured comparison and an explicit statement of the gap this project fills.

The five products were selected to cover the realistic range of choices a shop owner in this position would actually encounter: the two most internationally recognized POS platforms (Square, Loyverse), the most widely adopted global debt-ledger app (Khatabook), and the two categories of Arabic-market product most likely to appear in a local search for "best accounting app for shops" — a full Arabic accounting/ERP platform (Daftra) and a dedicated Arabic debt-documentation platform (Qayd).

2.1 International Point-of-Sale (POS) Platforms

2.1.1 Square Point of Sale

Square Point of Sale [1] is one of the most widely used commercial POS platforms internationally. How it works: a merchant registers items in a catalog with a price and stock count; at the time of sale, the cashier selects items, the app computes the total, and payment is taken by card or cash through Square’s own payment hardware and processing network; each sale is recorded as an independent, immediately-settled transaction, and reporting is built by aggregating these transactions.

Weaknesses relative to this project: Square has no concept of a service that occupies a physical, reusable slot for an extended period — such as a numbered shelf holding a customer’s device while it charges — because its entire data model assumes a sale is completed instantly at checkout. It also does not maintain a single running customer balance as a source of truth; instead of one number that rises and falls with debt and payment, it manages a series of separate invoices and payment records, which is a poor fit for a shop that extends informal credit to regular customers. Finally, Square charges recurring subscription

and per-transaction fees and offers no Arabic-first, low-bandwidth design suited to Gaza's environment.

Concretely, if this project's owner tried to use Square as-is for the charging shop, he would need to register each charging service (e.g., "phone charge," "power bank charge") as a catalog item, but would have no field to record which physical shelf a given device is sitting on, no way to mark a specific unit as "still charging" versus "already delivered," and would have to separately, manually track which regular customers currently owe money — defeating the purpose of moving away from a manual system in the first place.

2.1.2 Loyverse POS

Loyverse POS [2] works similarly to Square — a catalog-based sales and inventory system with a free base tier aimed at very small shops — using cloud sync so a merchant can review sales from any device. Its weakness for this project is identical to Square's: no shelf/device-occupancy concept, and no unified customer-balance model, only per-sale records. Its free pricing makes it more accessible than Square, but it solves the same narrow retail-sales problem and leaves debt tracking as an afterthought at best.

2.1.3 Feature-Level Comparison of the Two POS Platforms

Table (2.2) summarizes, at a finer level of detail than the overall comparison in Section 2.4, the specific features of Square and Loyverse that were examined during this review, to make concrete exactly which capabilities exist and which are absent.

Feature examined	Square	Loyverse
Barcode / catalog-item sale	Yes	Yes
Card payment processing	Yes (native)	Via third-party integration
Per-customer running balance	No (per-invoice history only)	No (per-invoice history only)
Physical shelf / slot tracking	No	No
Service-in-progress status (e.g. "charging")	No	No
Arabic interface	No	No
Free tier available	No	Yes, limited

2Table (2.2): Feature-level comparison of Square and Loyverse.

2.2 Global Digital Debt-Ledger Applications

A more directly relevant category of software replaces the paper "khata" (account book) long used by small merchants to track customer credit, without attempting to be a full POS system. Khatabook [3] is the most prominent global example. How it works: a shopkeeper creates one

digital ledger per customer and records two kinds of entries — money the customer owes (a sale on credit) and money the customer pays back — and the app automatically maintains and displays each customer's running balance, with optional SMS/WhatsApp payment reminders and cloud backup so records are not lost if the phone is lost or damaged. It has been adopted by tens of millions of merchants, which demonstrates that the specific problem of unreliable manual debt tracking is real and that shopkeepers will adopt a simple digital ledger to solve it.

Weaknesses relative to this project: Khatabook manages debt in isolation from everything else a shop does. It has no concept of inventory, no concept of a physical shelf or an in-progress service, and no way to register that a specific device is currently being charged for a specific customer. A shop using Khatabook to track debt would still need a completely separate system — paper, a spreadsheet, or a POS app — to track which devices are on which shelves and whether they have been collected. It also targets the Indian market first; while usable in Arabic-speaking regions, its interface and payment-reminder integrations (UPI, Indian mobile numbers) are not built around an Arabic, Gaza-specific context.

In practice, a charging-shop owner using only Khatabook would open the app to record that "Ahmad paid 5₪" or "Ahmad now owes 10₪," which is close to how this project's own deposit and debt-repayment screens work (Section 5.3), but would then need to separately remember, on paper or from memory, that Ahmad also has a phone currently charging on shelf 4 — reintroducing exactly the shelf-tracking problem (Section 1.1) that this project set out to solve.

2.3 Arabic-Market Accounting Platforms

2.3.1 Daftra

Daftra [4] is a cloud-based Arabic accounting and invoicing platform widely used by small and medium businesses across the Arab world, including Egypt and the Gulf states. How it works: a business subscribes and configures a full accounting suite — invoicing, expense tracking, inventory, payroll, tax settings per country, and customer-account management — accessed through a web dashboard and companion mobile apps for point-of-sale and quick expense entry. It automatically tracks customer balances and payment behavior as part of its broader accounting module, and its interface is fully Arabic and right-to-left, which is a genuine strength relative to Square and Loyverse.

Weaknesses relative to this project: Daftra is built as a comprehensive accounting and ERP-style platform — invoicing, HR and payroll, multi-branch inventory, cost centers, tax

compliance across multiple countries — which makes it powerful but heavy for a single small charging shop that needs to do one thing well: register a device on a shelf and track a simple running balance. Its pricing is subscription-based and scales with business size and feature set, and its setup process (chart of accounts, tax configuration, product catalog structure) assumes a level of formal bookkeeping structure most micro-shops, run by a single owner with no accounting background, do not need and are unlikely to configure correctly on their own. Like the other reviewed products, it also has no concept of a device occupying a numbered physical shelf; an owner would have to repurpose a generic inventory item to represent "a device currently charging," which the platform was never designed for and offers no dedicated screen or workflow to support.

2.3.2 Qayd

Qayd [5] is a Saudi Arabian digital platform built specifically around debt documentation and collection for small merchants — closer in spirit to Khatabook but designed natively for the Arabic market. How it works: a merchant records what each customer owes directly in Arabic, with the platform providing structured debt documentation intended to make claims easier to prove and follow up on, along with organized collection and reminder tools.

Weaknesses relative to this project: like Khatabook, Qayd is scoped narrowly to debt documentation itself. It does not manage inventory, retail sales, or — most importantly for this project — any notion of a physical shelf or an active service being rendered. A charging-shop owner using Qayd would still need a separate way to know which devices are currently occupying which shelves and whether a given device has already been paid for at delivery.

2.3.3 Feature-Level Comparison of the Two Arabic-Market Platforms

Table (2.3) similarly breaks down Daftra and Qayd at feature level, since both are Arabic-first but target different scopes of the problem.

Feature examined	Daftra	Qayd
Arabic RTL interface	Yes	Yes
Per-customer running balance	Yes (within full accounting module)	Yes (core purpose)
Retail sales / inventory	Yes (full module)	No
Physical shelf / slot tracking	No	No
Payroll, HR, multi-branch, tax compliance	Yes	No
Setup effort for a single micro-shop	High	Low
Free tier available	Limited trial	Freemium

3Table (2.3): Feature-level comparison of Daftra and Qayd.

Read together, Tables (2.2) and (2.3) make the same point from two different directions: the POS platforms are strong on sales and inventory but blind to balances and shelves, while the Arabic debt platforms are strong on balances but blind to sales, inventory, and shelves — and the one platform that covers both sales and balances (Daftra) does so at a cost of complexity poorly matched to a single micro-shop. None of the four addresses shelf or service-in-progress tracking at all.

2.4 Comparative Analysis

Table (2.4) compares the proposed system against the market solutions reviewed above, across the criteria most relevant to a charging shop’s actual daily operations.

Aspect	Proposed System	Square	Loyverse	Khatabook	Daftra	Qayd
Shelf / device-occupancy tracking	Built-in	No	No	No	No	No
Unified customer balance	Yes (single source of truth)	No (per-invoice)	No (per-invoice)	Yes	Yes (within full accounting)	Yes
Retail sales / inventory	Yes	Yes	Yes	No	Yes	No
Arabic RTL interface	Yes, tailored	No	No	Partial	Yes	Yes
Setup complexity for a micro-shop	Minimal	Moderate	Minimal	Minimal	High (full ERP)	Minimal
Cost	Free-tier cloud hosting	Subscription + fees	Free base tier	Free (freemium)	Subscription	Freemium/subscription
Multi-shop data isolation	Yes, enforced server-side	Yes (commercial)	Yes (commercial)	Yes (commercial)	Yes (commercial)	Yes (commercial)

4Table (2.4): Comparison between the proposed system and existing market solutions.

2.5 Why None of the Reviewed Products Was Adopted

Before this project began, the shop owner (Mohammed Eyad Abo Madkur, also a member of this development team) had already progressed through several of the record-keeping approaches described in Section 4.0 — paper notebooks, a general-purpose sales-recording app, and eventually a spreadsheet — without ever adopting a dedicated commercial product such as those reviewed in this chapter. Reviewing these five products in hindsight explains why: each one requires either abandoning a capability the shop actually needs (a POS platform with no balance concept, or a debt app with no shelf or inventory concept), or accepting a level of setup complexity and subscription cost (Daftra) disproportionate to a single-owner micro-shop’s needs. In other words, the market review in this chapter does not merely list competitors in the abstract; it explains, concretely, why the practical search described in Section 4.0 did not end at any of these products, and why it instead motivated building a system around the shop’s own workflow directly.

2.6 The Gap This Project Fills

This market review shows a consistent split. General POS platforms (Square, Loyverse) handle retail sales and inventory well but treat every sale as an instantly-completed transaction, with no concept of a customer balance as a single ongoing figure and no way to represent a physical slot — a shelf — occupied by an in-progress service. Debt-focused ledger applications (Khatabook, Qayd) solve the balance-tracking problem convincingly and, in Qayd’s case, natively in Arabic, but ignore inventory, retail sales, and physical resources entirely. Full Arabic accounting platforms (Daftra) come closest to combining these concerns, but as comprehensive ERP-style systems, they carry a level of setup complexity, subscription cost, and general-purpose accounting structure that does not match a single micro-shop’s actual daily workflow of receiving a device, placing it on a numbered shelf, and later delivering it back.

No reviewed solution combines a first-class shelf/device-occupancy model with a unified, automatically-derived customer balance, integrated retail sales, and a lightweight, Arabic-first, low-cost design built specifically for a single small charging shop. This combination — not any one feature in isolation — is the gap this project fills.

2.7 Theoretical Background

Beyond the market products reviewed above, this project also draws on established academic and technical literature that directly shaped specific architectural decisions.

REST Architecture. Fielding and Taylor [7] formalized the Representational State Transfer (REST) architectural style, built on resource-oriented URLs, a uniform set of HTTP methods, and statelessness. This principle was applied directly in designing the API’s endpoint structure (Table 5.1): every resource (customers, transactions, products, shelves) is addressed by its own URL, and the appropriate HTTP method (GET, POST, PUT, DELETE) determines the operation performed on it, rather than encoding the action into the URL itself.

ACID Transactions. Härder and Reuter [8] formally defined the ACID properties — Atomicity, Consistency, Isolation, and Durability — as the standard guarantee for reliable database transactions. This principle was applied directly in the `payDebt` and `deliverTransaction` functions (Sections 5.3, 4.6): every composite financial operation is

wrapped in a PostgreSQL transaction (BEGIN...COMMIT), ensuring that a deposit and its distribution across outstanding transactions either complete together or not at all, with no partially-applied state possible.

Multi-Tenant SaaS Design. Guo et al. [9] and Gao et al. [10] describe design patterns for multi-tenant SaaS applications, in which a single application instance serves many independent customers while keeping their data strictly isolated. This principle was applied directly in the authentication middleware (Section 5.2): every database query is scoped by an `owner_id` derived exclusively from the server-verified Firebase token, never from client input, enforcing the same isolation guarantee described in this literature.

Mobile Tools for Informal Economies. Toyama [11] studied mobile applications built specifically for informal and small-business economies in resource-constrained settings, finding that generic financial tools often fail to match these businesses' specific, informal workflows. This finding directly motivated the decision described in Section 4.0: rather than adopting a general-purpose accounting product, the system was built around the shop's own existing workflow (previously handled through Excel), reflecting the same insight that purpose-built tools succeed where generic ones fall short in this context.

Chapter 3__Methodology and Project Management

3.1 Methodology Name and Rationale

The project followed the Waterfall software development methodology, in which the work proceeds through a sequence of distinct phases — requirements, design, implementation, testing, and delivery — where each phase is completed before the next begins. This methodology was chosen because the system’s requirements were clear and well-defined from the start, derived from a well-understood workflow in an existing, operating shop.

Given the project’s size and the clarity of its goals, a structured sequential path was more suitable than an iterative one: the team could analyze the requirements fully, produce a complete design of the database and architecture, implement the system, and then test it, without the need for repeated requirement-gathering cycles.

The sequential, phase-gated nature of Waterfall, as originally formalized by Royce [6], made it a natural fit for a project whose requirements were already well understood from real, ongoing shop operations before development began.

3.2 Key Phases of the Methodology

- Requirements analysis: interviewing the shop owner and observing the actual daily workflow to produce a complete requirements specification.
- System design: modeling the database (ERD), the three-tier architecture, and the interfaces on Figma.
- Implementation: building the database, the REST API and its accounting logic, and the Flutter application.
- Testing: field-testing the completed system on the real shop’s operations and reconciling the resulting balances.
- Deployment and delivery: publishing the backend and database to the cloud and preparing the final documentation.

3.3 Adaptations to the Standard Methodology

The standard Waterfall model was followed with one practical adaptation: after the field-testing phase, a limited round of corrections was applied to fix issues discovered during real operation, without returning to a full new development cycle. This kept the process sequential while allowing the final product to be refined based on genuine feedback from the shop

owner, who acted as the on-site acceptance user. Version control through Git was used throughout to manage and integrate the work.

3.4 Tools and Equipment

- Flutter and the Dart language for the Android application.
- Node.js with the Express framework for the REST API.
- PostgreSQL as the relational database.
- Firebase Authentication for secure sign-in.
- Render for cloud deployment of the API and database.
- GitHub for version control, and Postman for API testing.
- Figma for interface and user-experience design.

3.5 Timetable

Table (3.1) shows the originally planned timetable.

Phase	Weeks	Deliverable
Requirements analysis	1–2	Approved requirements document
Design (ERD & architecture)	3–4	Database schema and interface designs
REST API development	5–7	Deployed API with unified accounting logic
Flutter app development	8–11	Complete application screens
Field testing & fixes	12–13	Test report and corrections
Documentation & delivery	14	Final report and presentation

Table (3.1): Planned project timetable.

Table (3.2) shows the actual timetable, reconstructed from the backend repository’s commit history, with each period mapped to its corresponding planned week from Table (3.1) for direct comparison.

Phase	Actual Period	Planned Week	Evidence	Deviation Reason
Initial setup, shelves, Firebase auth	May 2026 (exact start date not preserved in early commit history)	Weeks 1–7	Foundational API and authentication commits	None — broadly matched planned scope; sub-phase boundaries not tracked
Cleanup and debt/balance endpoints	Jun 3–Jun 9, 2026	Weeks 7–8	Debtors and customer-balance endpoints added	None — normal continuation of planned API work
Accounting logic rebuild (unified balance model)	Jul 11, 2026	Week 11 (planned: Flutter dev.)	Major refactor after identifying inconsistencies in the earlier design	Deviation: field testing revealed a design flaw in the original per-transaction payment-status model, requiring an unplanned backend rework (Section 3.7)
Deposits integration, dashboard balance	Jul 14–Jul 16, 2026	Weeks 11–12	Customer statement and dashboard updates	Direct consequence of the accounting rebuild above
Shelf multi-device fix	Jul 18, 2026	Weeks 12–13 (Field testing)	Bug found and fixed during field testing	None — exactly what the planned field-testing phase was designed to catch
Security cleanup & report finalization	Aug 2026	Week 14 (Documentation)	Credential removal, documentation	None — matches planned final phase; extended in duration due to additional report revisions requested during supervision

Table (3.2): Actual project timetable, mapped to planned weeks, with deviation reasons.

Comparing the two timetables shows that the overall sequence followed the planned Waterfall phases, but with one significant deviation: the accounting logic was substantially reworked in July after field testing revealed inconsistencies in the original per-transaction payment-status design, leading to the unified balance model described in Section 4.6. This is discussed further as a project risk in Section 3.7.

3.6 Team Management

3.6.1 Team Structure and Roles

- Mohammed Hisham Mahdi — UI/UX design of all screens on Figma and the visual identity.
- Mohammed Eyad Abo Madkur — database design, the unified balance accounting logic, and field testing.
- Mahmoud Mohammed Mady — REST API endpoints, authentication middleware, and cloud deployment.
- Hussein Ezzat Ghabayen — Flutter application development, state management, and API integration.

3.6.2 Communication and Collaboration

The team coordinated through instant messaging and regular meetings. Git and GitHub were used for version control throughout backend development: the API repository contains over 60 incremental commits spanning the full development timeline, from initial project setup and shelf-management logic, through the extended process of integrating Firebase Authentication, to the later rebuild of the accounting logic around the unified balance model, and finally the security cleanup performed before submission. The Flutter mobile application repository, in contrast, was consolidated and published to GitHub closer to the end of development, after most of its functionality had been implemented and tested locally.

3.6.3 Conflict Resolution and Delegation

Tasks were delegated by specialization as listed above. Technical disagreements were resolved through discussion and, where relevant, by validating options against the real shop data. Because responsibilities were clearly separated, integration conflicts were rare and were handled directly through Git.

3.7 Risks and Changes

The main risk that materialized during development was a design flaw discovered through field testing rather than upfront analysis: the original accounting design stored a separate payment status per transaction, which produced inconsistent results when a single payment settled multiple transactions. This was identified in early July 2026 during real operation in the shop and required a substantial rework of the backend's accounting logic (commit 0be77e9, "rebuild accounting logic - unified balance model"), replacing the per-transaction status approach with the unified customer-balance model described in Chapter 4. A secondary, related issue was discovered shortly after: when multiple devices shared a single shelf, delivering one device incorrectly freed the shelf and hid the remaining active devices; this was corrected on July 18, 2026 by making shelf release conditional on no other active transactions remaining on that shelf. Both issues illustrate a limitation of the Waterfall approach acknowledged in Section 3.3: because testing occurred after implementation rather than continuously, some design flaws were only caught once the system was exercised against real data.

Chapter 4__Requirements Analysis and System Design

4.0 Requirements Elicitation

Requirements for this system were not gathered through formal interviews with an external client. Instead, they emerged directly from the first-hand, practical experience of team member Mohammed Eyad Abo Madkur, who personally operates a mobile-charging shop. Several record-keeping methods were tried over time: purely paper-based logs, general-purpose sales-recording applications, and eventually spreadsheet software (Excel), which was structured closely around the shop's actual workflow and met roughly 80% of its day-to-day recording needs — tracking devices, sales, and customer balances reasonably well. However, Excel's remaining limitation was not technical but practical: it required a level of spreadsheet proficiency that made it inefficient, and effectively unusable, for anyone assisting in the shop who was not comfortable building and maintaining formulas. This motivated the core requirement behind the project: to preserve the same underlying record-keeping logic that Excel already captured, but present it through a simple, guided mobile interface — so that registering a transaction, searching a customer, and checking balances, profits, and sales could be done quickly and correctly by anyone working in the shop, not only someone skilled in spreadsheets. This lived experience directly shaped the functional requirements in Section 4.2, and the remaining gap between the Excel-based workflow and full digital fluency is also reflected in the future-work directions discussed in Chapter 8.

4.1 System Architecture

The system follows a three-tier architecture, as shown in Figure (4.1). The presentation tier is the Flutter Android application. The logic tier is a RESTful API built with Node.js and Express that contains all business logic and the accounting rules. The data tier is a PostgreSQL relational database. The application never accesses the database directly; every request travels over HTTPS to the API. Authentication is handled by Firebase: the application obtains a Firebase ID token (a JWT) and attaches it to every request, and a backend middleware verifies this token and maps it to the owner account through the `firebase_uid` field before any operation is executed. The API and the database are deployed on the Render cloud platform.

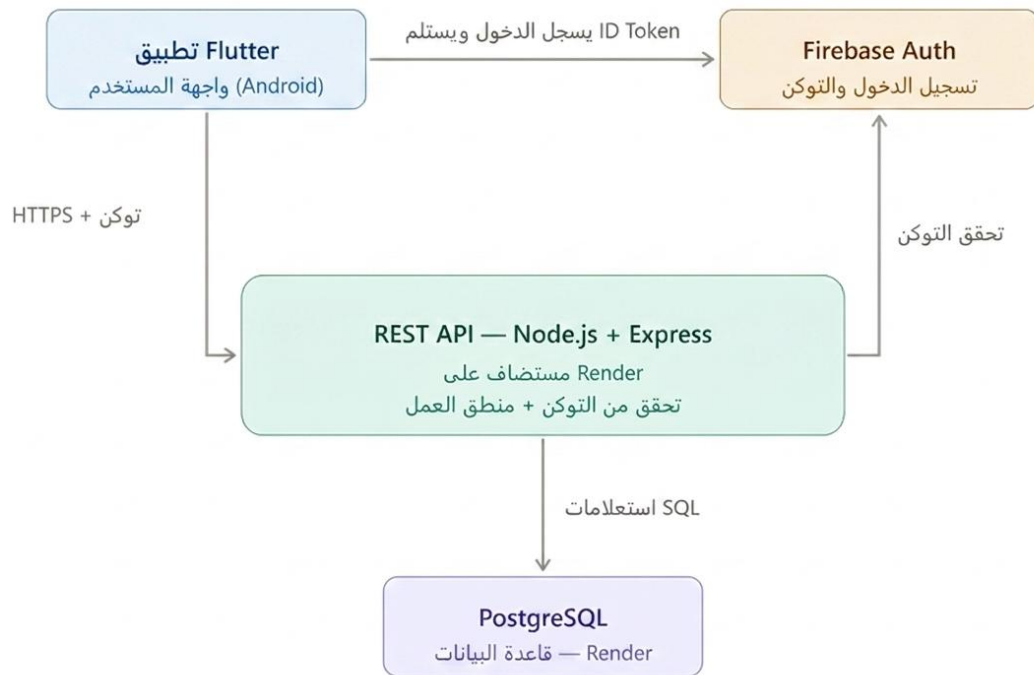


Figure (4.1): System architecture (three-tier)

4.2 Functional Requirements

Table (4.1) lists the functional requirements together with their acceptance criteria and the test case(s) in Chapter 6 that verify each one.

ID	Requirement	Acceptance Criteria	Verified By
FR-1	Owner signs in and accesses only their own data	Sign-in fails without a valid Firebase token; all queries return data for the signed-in owner_id only	Test T9 (Ch. 6)
FR-2	Register a charging device on a shelf with automatic price	Selecting a product auto-fills its price; device appears immediately as active on its shelf	T3 (device registration and price auto-fill), field-tested (Table 6.2a)
FR-3	Record a direct retail sale with quantity	Total = quantity × unit price, computed without manual entry	T1 (sale creation and balance deduction), field-tested (Table 6.2a)
FR-4	Deliver a device with optional payment	Delivering with full payment clears the related debt; delivering with 0 payment leaves balance unchanged	Test T4, T5 (Ch. 6)
FR-5	Record a deposit / debt repayment	Customer balance increases by exactly the deposited amount	Test T2 (Ch. 6)
FR-6	Maintain each customer's balance automatically	Balance always equals the sum of all transactions and deposits for that customer	Test T1, T2, T3 (Ch. 6)
FR-7	View a per-customer statement	Statement lists all transactions and deposits for that customer only	Test T6 (Ch. 6)
FR-8	View debtors list	Only customers with a negative balance appear	Test T7 (Ch. 6)

FR-9	View filtered operations log	Log correctly filters by day/week/month	Field-tested (Table 6.2a); no dedicated automated test case currently covers date-range filtering directly
FR-10	View live dashboard	Indicators update immediately after any transaction	Field-tested (Table 6.2a); dashboard values verified indirectly via T1–T3, T7 (Table 6.1)

7Table (4.1): Functional requirements, acceptance criteria, and traceability to testing.

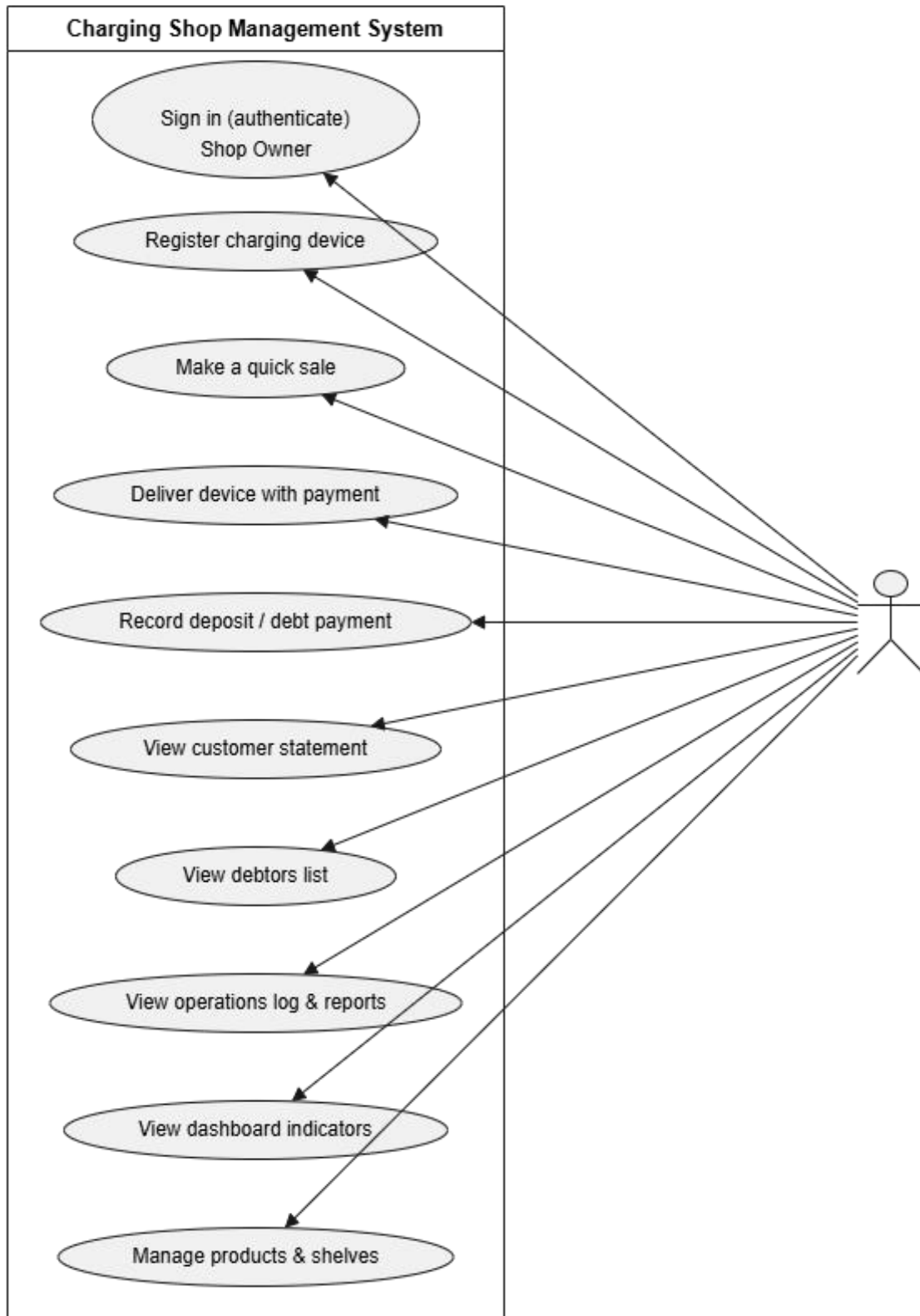
4.3 Non-functional Requirements

ID	Non-functional Requirement
NFR-1	Security — all requests are authenticated with a Firebase ID token verified server-side; each shop’s data is isolated via owner_id taken from the token, never from the client.
NFR-2	Reliability — composite financial operations use ACID database transactions to guarantee consistency.
NFR-3	Usability — a fully Arabic, right-to-left interface designed for a non-technical shop owner, minimizing the number of taps for frequent actions.
NFR-4	Performance — typical API responses return in well under a second when the cloud service is warm (see Chapter 6).
NFR-5	Portability & Availability — the backend and database are cloud-hosted, so the application works from any Android device with an internet connection.

8Table (4.2): Non-functional requirements summary.

4.4 Use-case Diagram

Figure (4.3) shows the main use cases for the single actor of the system — the Shop Owner — including registering a device, making a sale, delivering a device with payment, recording a deposit, and viewing statements, debtors, the log, and the dashboard. Customers do not interact with the system directly in the current scope (see Section 1.3); all operations are



2Figure (4.2): Use-case diagram.

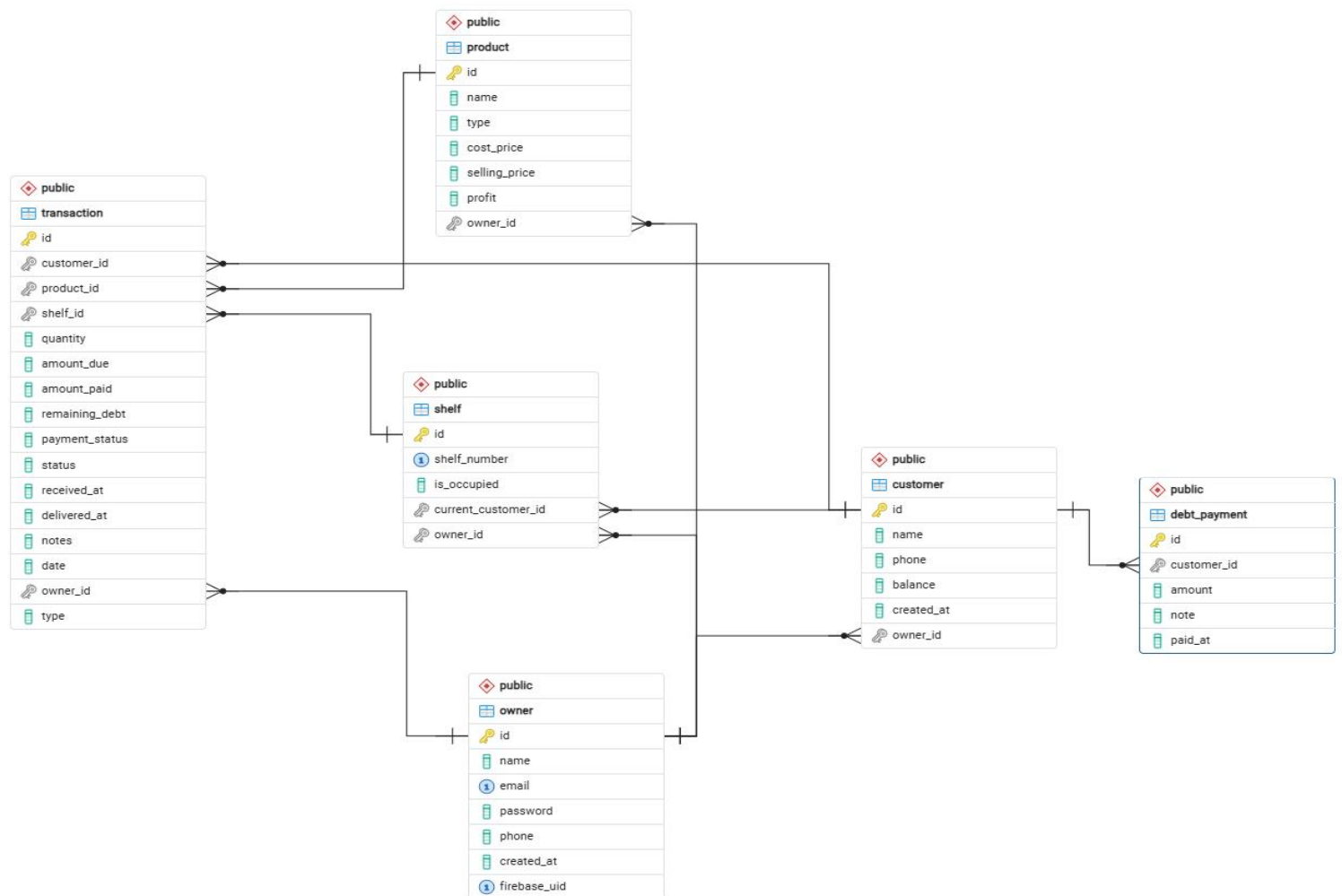
performed by the shop owner on their behalf.

4.5 Database Design

The database is structured around six related tables that represent the shop's owners, customers, products, shelves, and financial records, as shown in Figure (4.2). In every table, id is the Primary Key (PK).

- owner (PK: id): the shop owners; enables multi-tenant data isolation through owner_id, and links to Firebase via firebase_uid. This table also retains a legacy password column from an earlier design phase that predates the adoption of Firebase Authentication; it is not used by the current authentication logic, which relies exclusively on the verified Firebase ID token.
- customer (PK: id): customer information and the running account balance (name, phone, balance, owner_id, created_at).
- product (PK: id): the unified price list for both charging services and retail items (name, type, cost_price, selling_price, profit, owner_id).
- shelf (PK: id): the physical shelves and their occupancy for active charging devices (shelf_number, is_occupied, current_customer_id, owner_id).
- transaction (PK: id): the central record linking customer, product, and shelf for both charging services and retail sales (customer_id, product_id, shelf_id, quantity, amount_due, amount_paid, remaining_debt, payment_status, status, type, received_at, delivered_at, notes, date, owner_id).
- debt_payment (PK: id): every deposit or debt repayment made by a customer (customer_id, amount, note, paid_at). A charged device is represented as a transaction of type "charging" that references a shelf; delivering it marks the transaction as delivered and frees the shelf only when no other active device remains on it. This normalized design, together with foreign keys, keeps the data consistent by construction. All monetary columns (customer.balance, transaction.amount_due, transaction.amount_paid, transaction.remaining_debt, debt_payment.amount) are stored as double precision (a floating-point type) rather than a fixed-point NUMERIC type. The transaction.remaining_debt column is a PostgreSQL generated column, defined as GENERATED ALWAYS AS (amount_due - amount_paid) STORED: its value is recomputed automatically by the database whenever amount_due or amount_paid changes, is never written to directly by application code, and

therefore cannot become stale or inconsistent with the transaction it belongs to.



3Figure (4.3): Entity-Relationship Diagram (ERD).

4.6 The Unified Balance Accounting Model

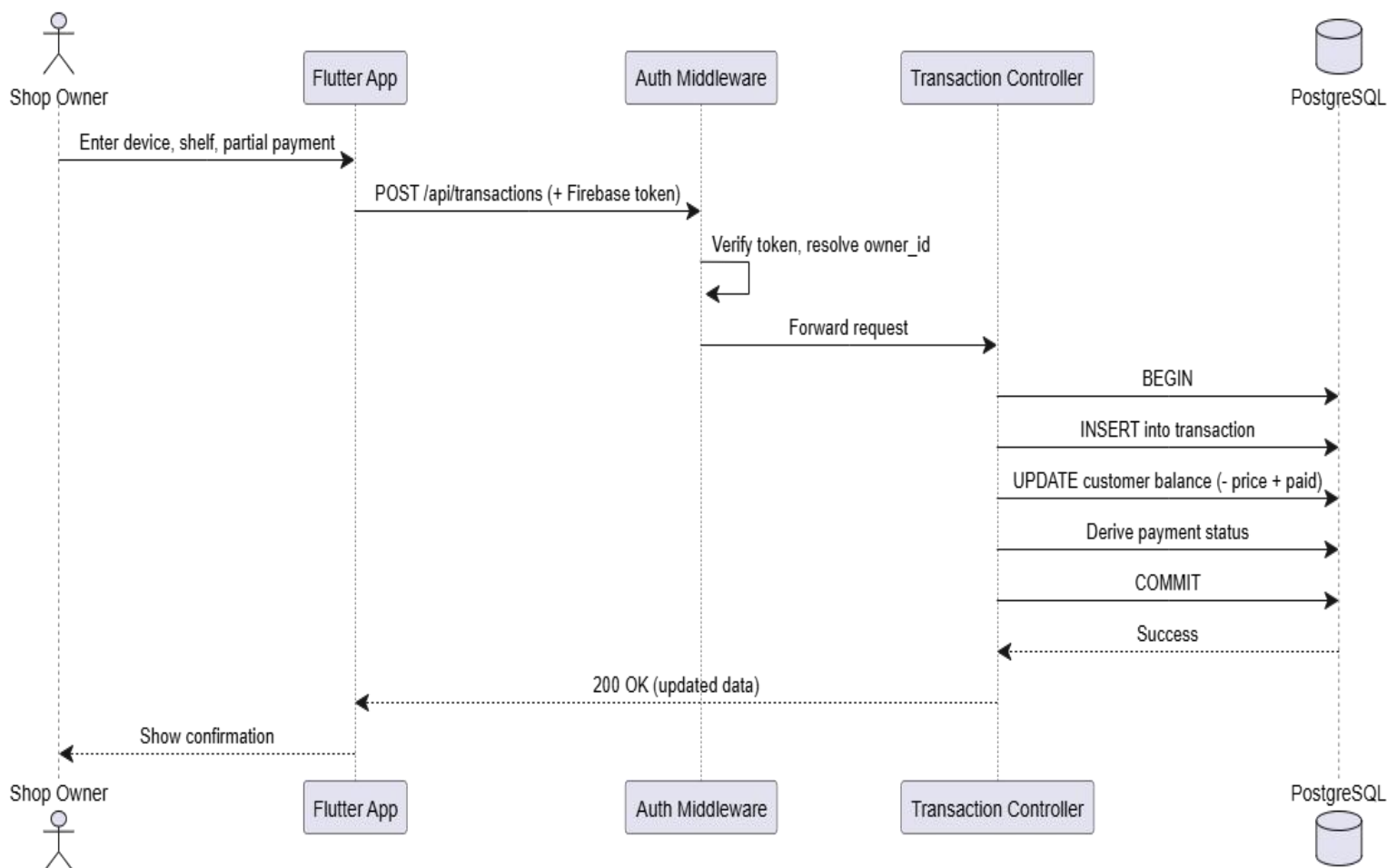
The heart of the design is a unified balance model in which the customer's balance column is the ultimate source of truth for a customer's financial position. Every transaction deducts its value from the balance and every deposit adds to it; a negative balance therefore means debt and a positive balance means credit held for the customer.

The transaction table also stores a `payment_status` field (e.g., paid, debt, partial) for each individual transaction. Unlike the customer's overall balance, this field is not a database-generated column: it is computed by the API's business logic at the moment a transaction is created or updated — comparing `amount_paid` against `amount_due` — and then persisted

alongside the transaction record. This design was chosen for read efficiency, since the shop's operations log and reports display many transactions at once. The customer's cumulative balance, however, remains the single authoritative figure for how much a customer currently owes or is owed, since it reflects the net effect of all their transactions and deposits over time, not just one transaction in isolation. Every composite financial operation (e.g., recording a deposit that updates both `debt_payment` and `customer.balance`) is executed inside an ACID database transaction, so it either completes entirely or not at all.

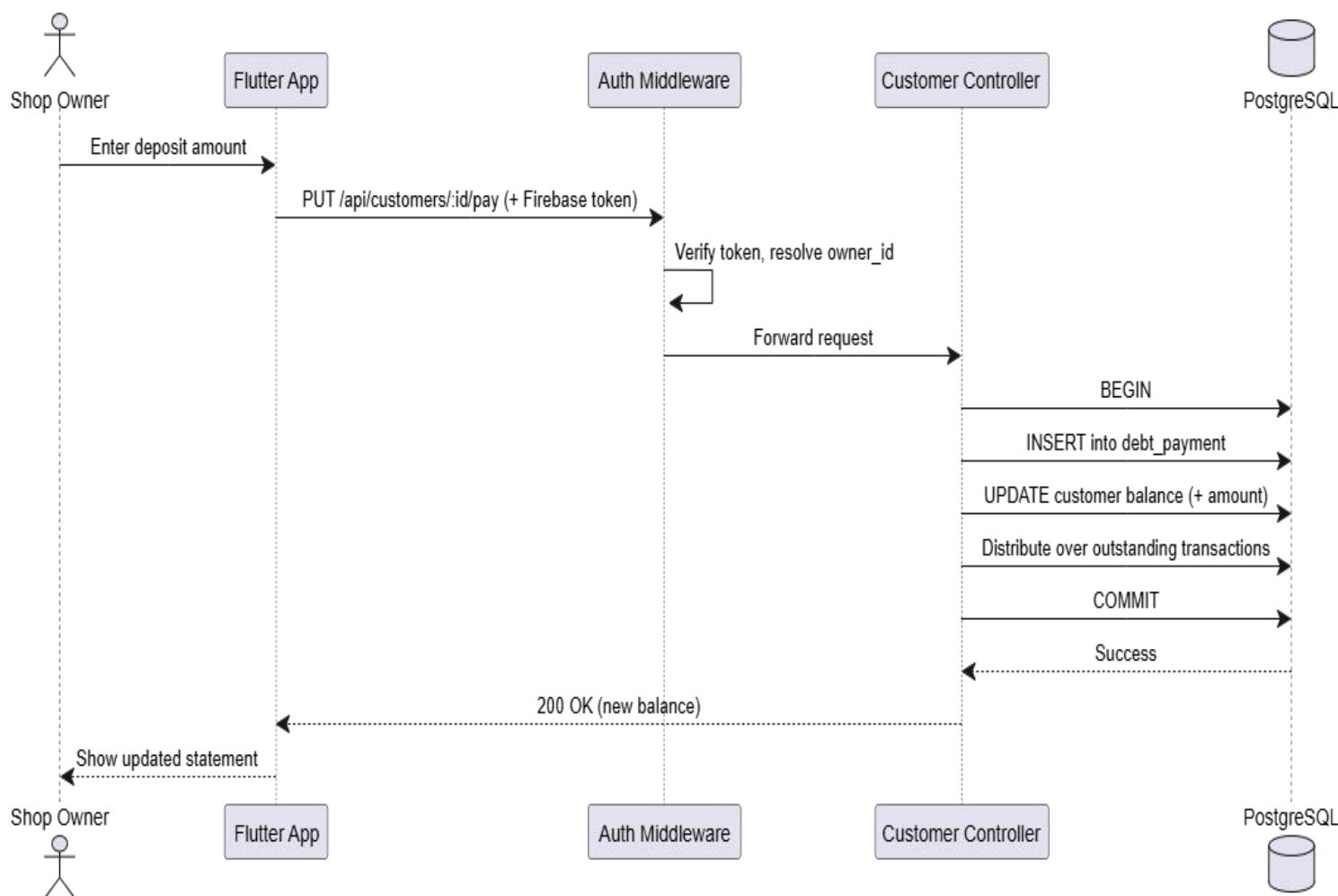
4.7 Sequence Diagrams

Figure (4.4) shows the sequence of adding a charging device with a partial payment: the application sends the request with the Firebase token; the middleware verifies it; the controller opens a transaction, inserts the transaction record, deducts the price and adds the paid amount to the customer balance, derives the status, commits, and returns the result.



4Figure (4.4): Sequence diagram — adding a device with a partial payment.

Figure (4.5) shows settling a customer debt: the application sends a deposit request; the middleware verifies the token; the controller opens a transaction, records the payment in `debt_payment`, adds the amount to the customer balance, distributes it across outstanding transactions for historical record, commits, and returns the updated balance.



5Figure (4.5): Sequence diagram — settling a customer debt.

Chapter 5__Implementation

5.1 Backend Organization

The backend is a RESTful API built with Node.js and Express, organized into three layers under `src/`: `routes/` (which HTTP method and path maps to which controller function), `controllers/` (the business logic and database access for each domain — customers, transactions, products, shelves, dashboard), and a single `middleware/` (Firebase token verification, applied to all protected routes). PostgreSQL is hosted in the cloud on Render alongside the API, with automatic redeployment on every git push.

5.2 Authentication and Owner Data Isolation

Authentication is implemented in `src/middleware/auth.js` and applied to every protected route. On each request, the middleware extracts the `Authorization: Bearer <token>` header, rejects the request with a 401 status if the header is missing or malformed, and otherwise verifies the token using the Firebase Admin SDK (`admin.auth().verifyIdToken(token)`). A successfully verified token yields the caller's `firebase_uid` and email.

The middleware then resolves this identity to a row in the `owner` table using `firebase_uid` or, as a fallback, the email address. If no matching owner exists, one is created automatically on first sign-in (a just-in-time provisioning pattern), so a new user does not need a separate registration step beyond signing in through Firebase; the `legacy_password` column is filled with a fixed placeholder value (`'firebase_auth'`) at this point rather than an actual credential, since it plays no role in verifying identity. The resolved owner record is attached to the request as `req.owner`, and every downstream controller uses `req.owner.id` as the `owner_id` for all database queries — this is the mechanism that enforces multi-tenant data isolation: `owner_id` is always derived from the verified token on the server side and is never accepted from client input. Expired or invalid tokens are rejected with a descriptive 401 response before any business logic executes.

5.3 Payment and Debt Processing

Payment and debt logic is implemented in `src/controllers/customerController.js`. Rather than repeating an `owner_id` filter on every statement, ownership is verified once per transaction through a "gate check": the initial `SELECT ... FOR UPDATE` query filters explicitly by both `id` and the authenticated `owner_id`, locking the row and returning a 404 immediately if no matching record exists for that owner. Statements that follow within the same database transaction — including the `UPDATE` to `customer.balance` and the `INSERT` into `debt_payment` — operate on that already-verified, locked row and therefore do not repeat the `owner_id` condition themselves; the `debt_payment` table (Section 4.5) has no `owner_id` column of its own, since its rows are scoped to a customer whose ownership was already confirmed in the same transaction. The one exception is the final `SELECT` that re-reads the customer after `COMMIT` to build the response: this read executes outside the locked transaction and does not itself re-check `owner_id`, relying instead on the `id` having already been validated earlier in the same request. This is a minor inconsistency in the current implementation, noted here rather than left for the committee to discover, and is listed as a hardening item in Section 7.7.

The `payDebt` function distributes each deposit across the customer's oldest outstanding transactions, selected via `WHERE remaining_debt > 0`. Because `remaining_debt` is a PostgreSQL generated column (Section 4.5), recomputed automatically as `amount_due - amount_paid` whenever either value changes, this filter always reflects each transaction's true outstanding balance at query time, with no risk of relying on a stale, manually-maintained value.

5.4 REST API Endpoints

Table (5.1) lists the main endpoints of the system.

Endpoint	Method	Purpose
<code>/api/customers</code>	GET	List all customers of the signed-in owner
<code>/api/customers/:id</code>	GET	Customer details: profile, transactions, deposits
<code>/api/customers/:id/pay</code>	PUT	Record a deposit / debt repayment (ACID)
<code>/api/customers/:id</code>	DELETE	Delete a customer (only if not in debt)
<code>/api/customers/payments/all</code>	GET	All deposits across customers
<code>/api/transactions</code>	POST	Create a charging or sale transaction
<code>/api/transactions</code>	GET	List transactions (with filters)
<code>/api/transactions/:id/deliver</code>	PUT	Deliver a device with optional payment
<code>/api/products</code>	GET / POST / PUT / DELETE	Manage the unified price list
<code>/api/shelves</code>	GET / POST	Manage shelves
<code>/api/dashboard</code>	GET	Live indicators and occupied shelves
<code>/api/dashboard/debtors</code>	GET	List of debtors and total debt

9Table (5.1): REST API endpoints of the system.

5.5 Frontend Organization

The Flutter application is organized under `lib/` into three layers: presentation/screens (the UI for each area — home, devices, debt, history), logic (one Cubit and one State class per

domain: customers, transactions, dashboard, products, shelves), and widgets (shared, self-contained components such as the add-device and quick-sale bottom sheets). State is managed with the Cubit pattern from `flutter_bloc`: each cubit exposes Loading, Success, and Error states, so every screen reacts consistently to the outcome of an API call instead of managing ad-hoc flags.

5.6 API Communication Layer

Each cubit communicates with the backend directly through the `http` package. Before every request, the cubit retrieves the current Firebase ID token via `FirebaseAuth.instance.currentUser?.getIdToken()` and attaches it as an `Authorization: Bearer <token>` header alongside `Content-Type: application/json`. The JSON response body is decoded and, based on the HTTP status code, mapped to either a `*Success` state carrying the parsed data or an `*Error` state carrying a message. In the current implementation this status check is binary (200 versus any other code), so network failures, expired-token (401) responses, and validation errors are all surfaced through the same generic error state rather than being distinguished for the user; refining this into more specific error handling is noted as an implementation improvement in Chapter 8.

The `CustomersCubit` illustrates this pattern: `getCustomers()` and `getCustomerDetails()` perform authenticated GET requests and emit `CustomersSuccess` or `CustomerDetailSuccess`; `payDebt()` performs an authenticated PUT to `/customers/:id/pay` and, on success, re-fetches the customer list so the interface reflects the updated balance immediately. Selected code from this cubit is provided in Appendix B.

5.7 Application Screenshots

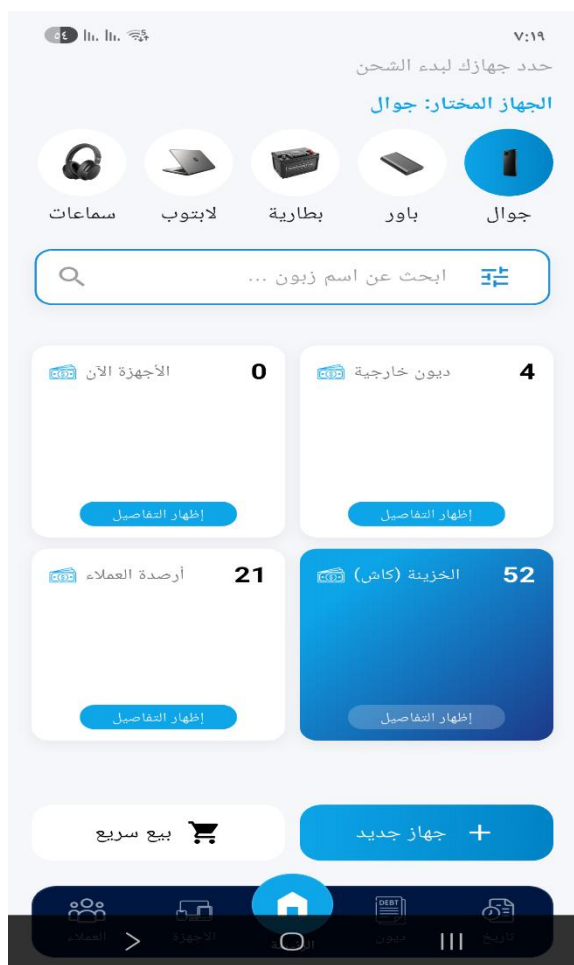
Figures below illustrate the core screens of the delivered application, corresponding to the functional requirements in Table (4.1). Each screen is described in terms of the endpoint that powers it and the logic that produces the values shown, so that its behavior can be verified against the implementation described earlier in this chapter.

5.7.1 Home Screen and Dashboard Indicators

The live dashboard shown in Figure (5.1) displays four indicator cards, each computed directly from the database on every request via `GET /api/dashboard` (Table 5.1), with no cached or separately stored values:

- الخزينة (Cash Received Today): the sum of amount_paid across all transactions dated today — `SUM(amount_paid) WHERE date = CURRENT_DATE`. This reflects today's actual cash inflow, not a running till balance.
- ديون خارجية (Outstanding Debts): the sum of the absolute value of every customer's balance where that balance is negative — `SUM(ABS(balance)) WHERE balance < 0` — i.e., the total amount currently owed to the shop across all customers.
- الأجهزة الآن (Active Devices): a count of shelves currently marked as occupied — `COUNT(*) FROM shelf WHERE is_occupied = true`.
- أرصدة العملاء (Customer Balances): the sum of every customer's balance where that balance is positive — `SUM(balance) WHERE balance > 0` — i.e., the total credit customers currently hold with the shop from overpayments or advance deposits.

All four indicators are recomputed live from the current state of the customer, transaction, and shelf tables on every dashboard request.



6Figure (5.1): Home screen and live dashboard

5.7.2 Registering a Charging Device on a Shelf

Figure (5.2) shows the device-registration flow, which implements FR-2. The owner selects a product marked as type = "charging" from the unified price list (Section 4.5), enters or selects a shelf number, and optionally records an initial payment. On submission, the app sends POST /api/transactions with the customer, product, shelf, and amount_paid. The controller creates a transaction row with amount_due set to the product's selling_price, marks the shelf as is_occupied = true with current_customer_id set to the chosen customer, and deducts (amount_due – amount_paid) from the customer's balance in the same database transaction. remaining_debt (Section 4.5) is then automatically available as a generated column reflecting any unpaid portion.

The screenshot displays a mobile application interface for registering a new charging device. The main form is titled "تسجيل جهاز جديد" (Register New Device) and includes the following fields and options:

- اسم العميل** (Customer Name): A text input field containing the name "ساهر".
- رقم الرف** (Shelf Number): A text input field containing the number "1".
- نوع الجهاز / الخدمة** (Device Type / Service): A dropdown menu currently showing "باوربانك - 2" (Powerbank - 2).
- السعر** (Price): A text input field showing "2.00" with a currency symbol (AED).
- المدفوع الآن** (Amount Paid Now): A text input field showing "0.00" with a currency symbol (AED).
- Payment Options:** Two buttons are visible: "دفع كامل (2)" (Full Payment (2)) in green and "لم يدفع (على الحساب)" (Did not pay (on account)) in orange.
- Submit Button:** A large blue button labeled "حفظ الجهاز" (Save Device).

The background of the app shows a list of devices and a bottom navigation bar with icons for home, search, and other functions.

Figure (5.2): Registering a charging device on a shelf

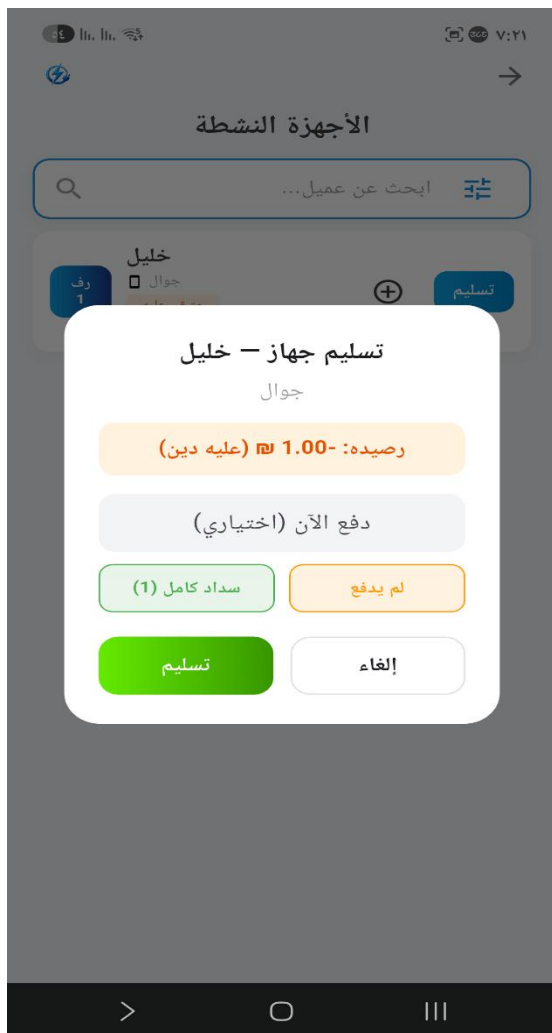
5.7.3 Quick Sale of a Retail Item

Figure (5.3) shows the quick-sale flow, implementing FR-3. The owner selects a product of type = "sale", enters a quantity, and the app computes the total locally as quantity × selling_price before submission. This total is sent as amount_due in the same POST /api/transactions call used for charging devices (Section 5.7.2), distinguished only by type = "sale" and the absence of a shelf_id — no shelf is occupied for a retail sale. As with device registration, any amount not paid immediately is deducted from the customer's balance, and the sale appears in the operations log (Section 5.7.7) alongside charging transactions.

8Figure (5.3): Quick sale of a retail item

5.7.4 Smart Delivery with Payment Collection

Figure (5.4) shows the delivery flow, implementing FR-4. When the owner taps "Deliver" on an active device, the app displays the customer's current overall balance (Section 4.5) — not just the amount due on this single transaction — so the owner can see the customer's full financial position before deciding how much to collect. The owner may enter a full, partial, or zero payment. On confirmation, the app sends PUT /api/transactions/:id/deliver; the controller marks the transaction as delivered, adds any collected payment to the customer's balance, and frees the shelf (is_occupied = false) only if no other active transaction remains on that shelf (Section 4.5) — the fix for the multi-device shelf issue discussed in Section 3.7.



9Figure (5.4): Smart delivery with payment collection

5.7.5 Customer Statement

Figure (5.5) shows a customer's unified statement, implementing FR-7, generated by GET /api/customers/:id (Table 5.1). The screen merges two data sources into one chronological list: the customer's transactions (charging and sales) and their deposits (debt_payment table, Section 4.5), alongside the customer's current balance at the top.

To illustrate exactly how the displayed balance is derived, consider the customer shown in Figure (5.5), who has a balance of 5.00: two transactions recorded as "debt — paid 0.00" (a 1₪ charging transaction and a 4₪ sale, both currently unpaid, totaling 5₪ owed) and one deposit of 10₪. Applying the unified balance model (Section 4.6): starting balance 0, minus 1 (charging transaction) minus 4 (sale) plus 10 (deposit) = 5.00, matching the displayed figure exactly. The two transactions still display "debt — paid 0.00" individually because, per the distribution logic in payDebt (Section 5.3), the 10₪ deposit was recorded before these two transactions existed and therefore could not be distributed against them retroactively; only deposits made after a transaction exists are distributed against it. The customer's overall balance nonetheless correctly reflects their true position, since it is the sum of all transactions

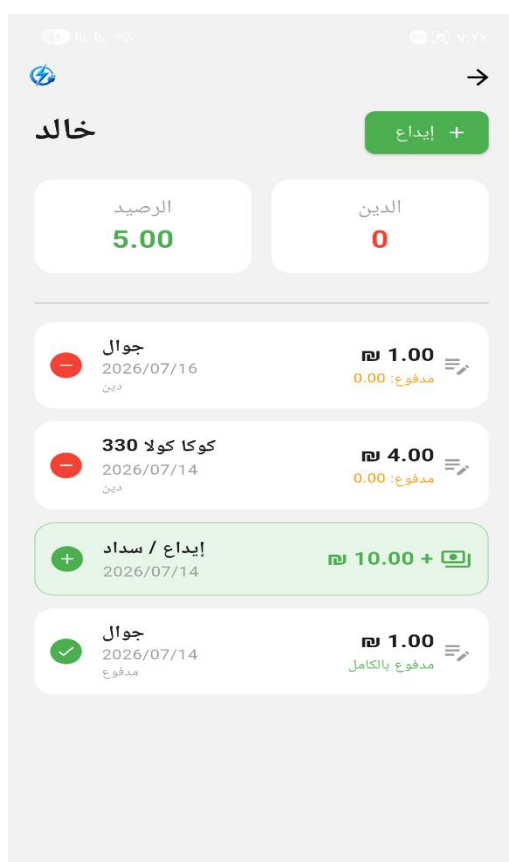
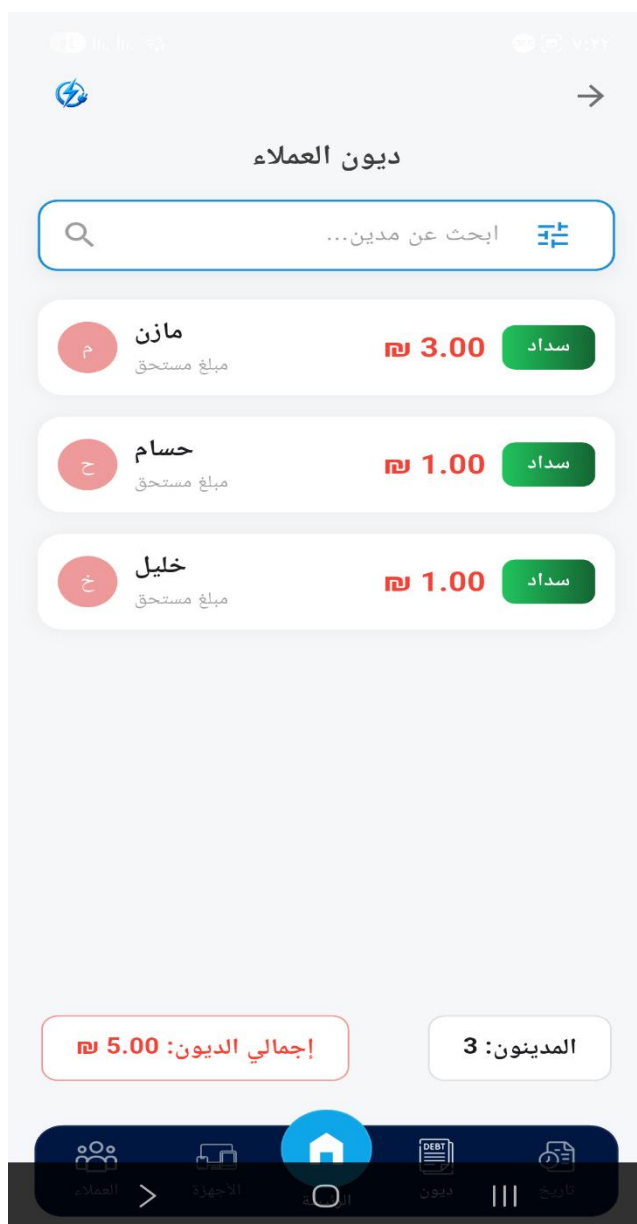


Figure (5.5): Customer statement

and deposits regardless of the order in which individual transaction statuses were updated.

5.7.6 Debtors List

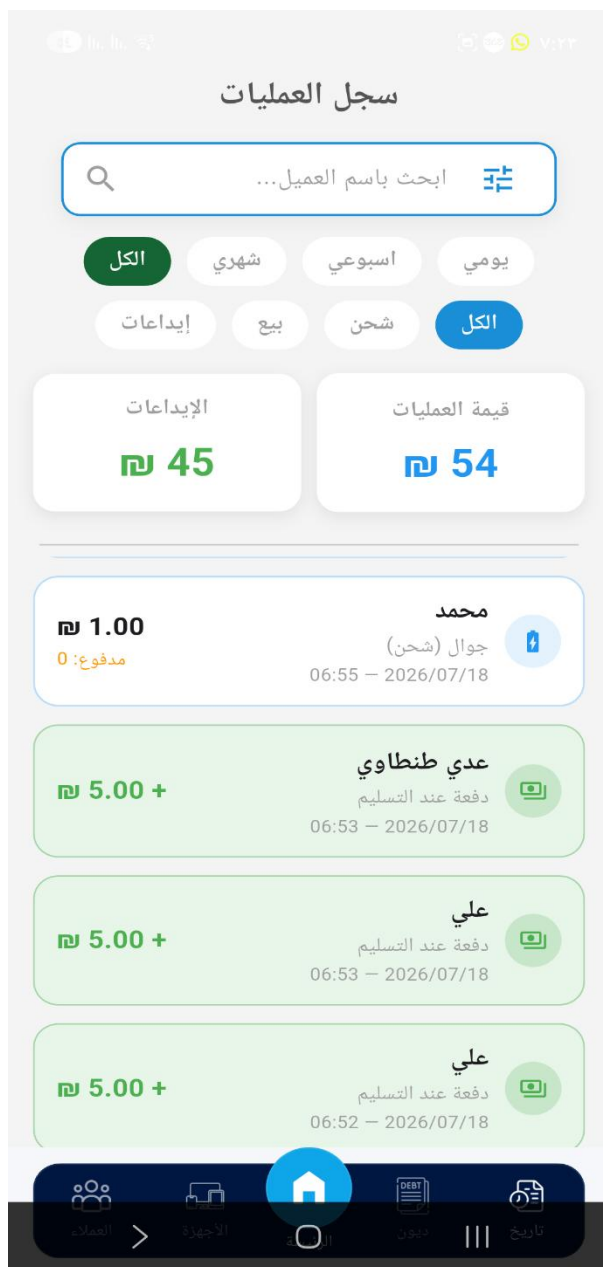
Figure (5.6) shows the debtors list, implementing [FR-8](#), generated by GET `/api/dashboard/debtors` (Table 5.1). The query (Section 5.1) filters the customer table to WHERE balance < 0, so customers with a zero or positive balance never appear here regardless of their transaction history. Each row displays the customer's name and the absolute value of their balance as the amount owed; the total shown at the bottom of the screen is the sum of these values, matching the "ديون خارجية" dashboard indicator (Section 5.7.1) computed by the same underlying logic.



11Figure (5.6): Debtors list

5.7.7 Filtered Operations Log

Figure (5.7) shows the filtered operations log, implementing FR-9, generated by GET /api/transactions with query parameters for the selected period. The owner can filter by day, week, or month; the app sends the corresponding date range to the API, which filters the transaction table on the date column (Section 4.5) and returns matching rows regardless of type (charging or sale) or payment_status, so the log reflects every recorded operation in the selected period, not only fully paid ones.



5.8 Deployment

The API and the PostgreSQL database are deployed on the Render cloud platform, with automatic redeployment triggered on every push to the GitHub repository. As a characteristic of the free hosting tier, the service enters a sleep state after a period of inactivity; the first request after sleep therefore incurs a cold-start delay (measured at about 22 seconds), after which subsequent requests respond normally (a median of about 552 ms). This behavior and its mitigation are discussed in Chapters 6 and 8.

Chapter 6__Testing and Evaluation

6.1 Testing Strategy

The system was tested at three levels. First, functional testing of the API endpoints was performed with Postman to confirm that each operation produces the correct result and updates the customer balance correctly, including edge cases such as invalid input and cross-shop access attempts. Second, non-functional behavior was verified under adverse conditions, including network disruption and the cloud service's cold-start delay. Third, user acceptance testing (UAT) was carried out inside the real shop, where the system was operated on genuine daily transactions and the resulting accounts were reconciled against manual expectations. A Postman collection reproducing all functional test cases is provided alongside the source code (Section 6.5).

6.2 Test Case Specification

Table (6.1) specifies representative test cases, each traced to the functional or non-functional requirement it verifies (Chapter 4).

ID	Req.	Objective	Input	Steps / Expected	Actual	Status
T1	FR-6	Balance deduction on unpaid sale	Sale 4₩, paid 0	POST /transactions → balance = -4	-4	Pass
T2	FR-5,6	Deposit increases balance	Deposit 5₩	PUT /customers/:id/pay → balance = +1	+1	Pass
T3	FR-2,6	Charging device deducts price	Device 1₩, paid 0	POST /transactions → balance -1	-1	Pass
T4	FR-4	Deliver without payment	paid = 0	deliver → no balance change; shelf freed if empty	Balance unchanged (remained at prior value); shelf status confirmed unoccupied only after last device removed	Pass
T5	FR-4	Deliver with full payment	Full amount	deliver → debt cleared, deposit recorded	Debt cleared to 0; new debt_payment row created with amount equal to the transaction's remaining_debt	Pass
T6	FR-7	Statement returns full history	—	GET /customers/:id → profile+tx+payments	All present	Pass
T7	FR-8	Debtors list excludes non-debtors	—	GET /dashboard/debtors → negative balances only	Debtors only	Pass
T8	§4.5	Multi-device shelf handled correctly	2 devices, 1 shelf	Deliver one → second stays; shelf still busy	First device marked delivered; shelf remained is_occupied = true; second device still visible in active-devices list	Pass
T9	NFR-1	Reject unauthenticated requests	Missing/invalid token	Any protected endpoint → 401	401 returned	Pass
T10	NFR-1	Cross-shop data isolation	Owner A token, Owner B customer_id	GET /customers/:id → 404 (not visible)	404 returned	Pass
T11	Input val.	Reject negative deposit	amount = -10	PUT /customers/:id/pay → 400	400 returned	Pass
T12	Input val.	Reject zero deposit	amount = 0	PUT /customers/:id/pay → 400	400 returned	Pass
T13	FR-4,6	Partial payment updates balance/status	due=10, paid=6	deliver → remaining -4; status=partial	remaining_debt updated from 4 to 0 automatically (generated column); payment_status changed from "partial" to "paid"	Pass
T14	NFR-5	App behavior on network disruption	Airplane mode mid-request	Any API call → app fails safely, no crash	Generic error shown (see note)	Pass

T15	NFR-4	Cold-start recovery	First request after ~15 min idle	Any GET endpoint → eventually succeeds	Succeeded after ~22s	Pass
T16	NFR-2	Concurrent deposit handling on same customer	Two different-amount deposits, mobile app + Postman, near-simultaneous submission	Both requests should succeed; final balance = sum of both amounts; no lost update	Both succeeded; balance correctly reflected sum of both amounts; two separate debt payment records created	Pass

10Table (6.1): Test case specification, traced to functional and non-functional requirements.

* T14: the current error-handling implementation (Section 5.6) does not yet distinguish a network disruption from a 401 or a validation error at the UI level; the test confirms the app fails safely (no crash), but this remains a noted area for improvement (Chapter 8).

6.3 User Acceptance Testing (UAT)

UAT was conducted with two deliberately separate groups, because the development team is not a substitute for an external user's perspective on a system they did not build.

6.3.1 Field Usage by Shop Staff

Two of the three daily users of the system are external to the development team: the two brothers of team member Mohammed Eyad Abo Madkur, who work in the shop and handle its daily operations but have no role in building the system. Mohammed Eyad Abo Madkur himself — introduced in Section 2.5 as both a development team member and the operator of the mobile-charging shop — is also the system's primary daily user in a real operational capacity, a dual role disclosed here explicitly rather than blurred. His usage is included in this table because it reflects genuine, unscripted daily operation of the shop, distinct from the scripted functional testing performed by the wider team in Section 6.3.2.

Item	Description
Participants	3 — two external users (the shop owner's brothers, who handle daily operations and have no role in development) and the shop owner himself
Tasks	Register a device, quick sale, deliver with payment, record a deposit, review statement/debtors/log, use the dashboard — performed as part of real, ongoing daily work
Environment	Live production system, real daily shop operations
Result	All core tasks completed successfully in normal daily use; balances matched manual expectations; the interface was reported as faster and less error-prone than the previous Excel-based process, including by staff with limited spreadsheet skills

11Table (6.2a)Field usage by shop staff

6.3.2 Internal Team Members — Technical/Functional Testing

Item	Description
Participants	4 — the development team members
Tasks	Repeating the core tasks above plus the edge cases in Table (6.1) (T10–T15) not naturally occurring in daily operation
Result	All tasks completed successfully; this group's results are treated as functional verification, not as external usability validation
Note	A formal large-scale load/stress test with many simultaneous external users was not conducted and is listed as future work (Chapter 8)

12Table (6.2b): UAT — internal team members.

6.4 Performance Measurement (Revised)

API latency was measured manually using Postman against two endpoints (sign-in and GET /api/customers) across four distinct connectivity states, with 24 total samples collected rather

than a single warm/cold pair. Measurements were taken on a standard broadband connection in August 2026. Table (6.3) reports the observed response times for each state.

Condition	N	Observed Response Time(s)	Note
Full cold start (after idle, service suspended)	2	≈ 22 s; 23.70 s	Two independent measurements on different endpoints, consistent with each other
Warming up (first 1–2 requests immediately after cold start)	4	1.55 s, 1.23 s, 3.65 s, 1.13 s	Response time decreasing rapidly across successive requests; magnitude varies by endpoint but the warm-up pattern is consistent
Fully warm (20 rapid successive requests, GET /api/customers)	20	Min 229 ms — Max 605 ms	All 20 samples fell within this range; typical response well under 1 s
Partially cooled (brief pause, not a full idle timeout)	1	> 1.5 s	Slower than fully warm but faster than a full cold start; reflects partial connection/process reuse loss

13Table (6.3): API response time across connectivity states (Postman; sign-in and GET /api/customers endpoints; Aug. 2026; broadband connection; 24 total samples).

Note: the 20-sample fully-warm measurement provides a genuine min–max range; a formal median and 95th-percentile calculation across a larger, automated sample was not performed and remains listed as a testing improvement in Section 7.7.

These results show that NFR-4 ("typical API responses return in well under a second when the cloud service is warm", Section 4.3) is met once the service is fully warm: across 20 consecutive requests, response time ranged from 229 ms to 605 ms, entirely within the sub-second target. The results also confirm that "warm" is not a single fixed state: the service passes through a brief warming-up period (roughly 1–4 seconds) immediately after waking, and can partially cool again after only a short pause between requests. The full cold start, measured independently on two occasions (≈22 s and 23.70 s), remains the worst case and occurs only after an extended idle period; it is mitigated as discussed in Sections 7.6 and 8.2 (keeping the service warm or upgrading the hosting tier).

6.5 Postman Collection

A Postman collection covering every endpoint listed in Table (5.1), including the functional and edge-case test scenarios in Table (6.1), is provided alongside the source code in the charging-api repository (see Appendix A), so the results reported in this chapter can be independently reproduced against the deployed API.

6.6 Evaluation Against Objectives

While Table (6.1) verifies functional and non-functional requirements (FR/NFR), it does not directly evaluate the five measurable objectives defined in Section 1.2.2. Table (6.4) closes this gap by evaluating each objective (O1–O5) directly against the specific numeric criterion

promised for it, describing how that criterion was measured, and reporting the observed result honestly — including partial achievement where applicable.

ID	Numeric Criterion	How Measured	Evidence	Observed Result	Status
O1	Zero manual price entries	Manually attempted device/sale registration and checked whether price required manual input	Section 5.7.2, 5.7.3 (implementation); field usage (Table 6.2a)	Price auto-filled from the product table on every registration; no manual entry required in any observed case	Achieved
O2	100% of payment statuses derived automatically	Inspected the transaction and customer tables directly (Section 4.5, 4.6) for any manually-set status field	remaining_debt confirmed as a PostgreSQL GENERATED column (Section 4.5); T1–T3, T13 (Table 6.1)	payment_status is always computed by application logic or the generated column; no status field is ever set directly by the client	Achieved
O3	Shelf/device state reflected within a single API call	Inspected the GET /api/dashboard response for shelf state after a delivery	Section 5.1; T8 (Table 6.1)	Occupied/free shelf state returned correctly in one call; T8 confirms correct state after multi-device delivery	Achieved
O4	≤3 taps from home screen	Manually counted taps on the final build, starting from the home screen, for each of the three target screens	Figures 5.5, 5.6, 5.7 (customer statement, debtors list, filtered log)	Customer statement: 2 taps. Debtors list: 1 tap. Filtered (monthly) log: 2 taps. All three within the ≤3 target	Achieved
O5	Complete data isolation, verified by dedicated test cases	Executed a cross-shop access attempt using one owner's token against another owner's customer record	T10 (Table 6.1)	Request returned 404 Not Found; no cross-shop data was returned	Achieved

14Table (6.4): Evaluation of each measurable objective against its specific numeric criterion.

All five objectives are achieved against the numeric criteria set for them in Section 1.2.2, each backed by a specific, reproducible measurement rather than a general claim: O1–O3 and O5 through direct inspection of the implementation and dedicated test cases (Table 6.1), and O4 through a manual tap count performed on the final build, reported per screen in Table (6.4) rather than as a single aggregate figure.

6.7Concurrency Behavior

The payDebt function (Section 5.3) locks the target customer row using SELECT ... FOR UPDATE before modifying its balance, which should serialize concurrent requests at the database level rather than allowing them to interleave. This was verified directly (Test T16, Table 6.1): two deposits of different amounts were submitted for the same customer from two separate clients — the mobile application and Postman — triggered as close to simultaneously as manually achievable. Both deposits completed successfully, and the customer's final balance correctly reflected the sum of both distinct amounts, with two separate records created in debt_payment and no evidence of a lost update or corrupted balance. This confirms, through direct concurrent testing rather than code analysis alone, that the ACID transaction guarantees described in Section 4.6 hold when two clients attempt to modify the same customer's balance at the same time.

Chapter 7__Deployment, Security, and Limitations

7.1 Deployment Architecture

The system's three cloud services are described below, corresponding to the architecture in Figure (4.1).

- **Render:** hosts both the Node.js/Express API and the PostgreSQL database, on Render's free tier. The API is configured for automatic redeployment on every push to the main branch of the charging-api GitHub repository.
- **Firebase:** provides Authentication only (email/password sign-in and ID token issuance and verification); no application data is stored in Firebase — all business data resides in PostgreSQL.
- **PostgreSQL:** the single relational database for all six tables described in Section 4.5, hosted on Render alongside the API rather than on a separate database provider.

7.2 Cost

The system currently runs entirely on free-tier services: Render's free web-service and free PostgreSQL instance, and Firebase Authentication's free usage tier, which comfortably covers a single shop's request volume. This keeps the running cost at zero for the current single-shop deployment. Render's free PostgreSQL instances are time-limited and expire after a fixed period (90 days at the time of writing) unless upgraded to a paid plan, and the free web-service tier is what causes the cold-start behavior discussed in Section 7.6; both are direct cost/performance trade-offs of operating at zero cost, and are revisited in Chapter 8 as a consideration for scaling to multiple paying shops.

7.3 Backup Policy

At present, the system relies on Render's underlying infrastructure for database durability, and no separate, project-managed backup procedure (e.g., scheduled exports of the `debt_payment` and transaction tables) has been implemented. This is a limitation rather than a deliberate design choice: for a system whose core value is an accurate, trustworthy debt ledger, the absence of an independent backup routine is acknowledged here explicitly, and periodic export of financial tables is listed as a near-term improvement in Chapter 8.

7.4 Secrets Management

Database credentials and the Firebase service-account configuration are supplied to the API through environment variables rather than being hard-coded in the source. During development, several standalone utility scripts that embedded the database connection string

directly were mistakenly committed to the charging-api repository; this was identified and corrected by removing the scripts from version control and adding them to .gitignore. This incident is noted here transparently as a lesson learned in secrets management, and an .env.example file (with placeholder values only) is provided in the repository to guide correct configuration going forward (see Appendix A).

7.5 Transport Security and Authentication Lifetime

All communication between the Flutter application and the API occurs over HTTPS, as Render terminates TLS for all deployed services by default; no endpoint is served over plain HTTP. Authentication uses Firebase ID tokens, which are short-lived (approximately one hour) and are refreshed transparently by the Firebase client SDK on the mobile application; the application requests a fresh token before each API call via `FirebaseAuth.instance.currentUser?.getIdToken()` rather than caching a single long-lived token, so an expired token results in automatic renewal rather than requiring the user to sign in again under normal conditions.

7.6 Fault Handling

Two categories of fault were specifically tested, as reported in Chapter 6. First, cold-start recovery (Test T15): after a period of inactivity, Render’s free tier suspends the service, and the first subsequent request incurs a delay of approximately 22 seconds before the service responds; once warm, the service responds normally (median ≈ 552 ms, Table 6.3). Second, network disruption (Test T14): if a request fails due to connectivity loss, the mobile application catches the exception and displays a generic error state rather than crashing; as noted in Section 5.6, this generic handling does not yet distinguish a network failure from other error types (e.g., 401 or 400 responses), which is listed as an implementation improvement in Chapter 8.

7.7 Current Limitations

The following limitations are stated explicitly, reflecting the actual state of the deployed system rather than its intended future scope:

- Internet dependency: the application requires an active internet connection for every operation; there is no offline mode. Given that intermittent connectivity is part of the environment that motivated this project, this is considered the most significant current limitation, and an offline-first design with local caching and background synchronization is prioritized as future work (Chapter 8).

- No independent backup routine for financial data beyond the hosting provider's infrastructure (Section 7.3).
- Generic error handling that does not yet distinguish network failures, authentication failures, and validation failures at the user-interface level (Section 7.6).
- No formal load or stress testing has been conducted; the system's behavior under many simultaneous users or high transaction volume is untested (Section 6.3.2).
- Free-tier hosting introduces a cold-start delay on the first request after idle periods (Section 7.6), and free-tier database instances are time-limited (Section 7.2).
- Not every read statement re-verifies `owner_id` independently; the final customer read in `payDebt` (Section 5.3) relies on the id having been validated earlier in the same request rather than repeating the check — a hardening opportunity rather than a currently-exploitable gap, since the id is never accepted from an unauthenticated source.
- No automated unit tests exist for the backend or mobile application; all testing described in Chapter 6 was performed manually through Postman and direct field use, with no continuous integration (CI) pipeline currently configured to run tests automatically on each code change.
- The Flutter application calls the REST API directly from each Cubit (Section 5.6) without an intermediate repository or service layer; this keeps the codebase simple for the project's current size but would make future changes to the API communication logic (e.g., switching HTTP clients or adding request caching) require updating each Cubit individually rather than a single shared layer.

7.8 Customer Data Protection

The system collects and stores a limited set of personal data about each customer: name, phone number, running account balance, and a chronological history of their transactions and deposits (Section 4.5). No customer passwords, government identifiers, or payment card details are collected or stored — the only authentication credentials in the system belong to shop owners, verified through Firebase (Section 5.2).

This data is accessible exclusively to the shop owner whose customers it belongs to: every query is scoped by `owner_id` derived from the owner's verified authentication token (Section

5.2), so no other shop owner using the same deployed system can view or query another shop's customer records. Within the mobile application itself, any staff member with access to the owner's signed-in device can view this data, as the current design does not implement separate staff-level accounts or permissions (noted as a limitation in Section 7.7).

Customer data is retained indefinitely by default, for as long as the shop owner's account remains active, since an ongoing debt or credit relationship with a customer may span months or years and the owner needs continuous access to this history. There is currently no automated data-retention limit or scheduled deletion policy.

A customer record can be deleted on request through the `deleteCustomer` function (Section 5.3), which permanently removes the customer along with their associated transaction and `debt_payment` records from the database. As a safeguard, deletion is blocked while the customer's balance is negative (i.e., while they are in debt), so a customer's financial history cannot be erased while an unresolved debt exists; this check is enforced directly in the database transaction (Section 5.3). Because Render's underlying infrastructure is the only backup mechanism currently in place (Section 7.3), a deletion is not independently recoverable once committed.

Chapter 8__Conclusions and Future Work

8.1 Conclusions

This project set out to replace the manual, error-prone management of mobile-charging shops with a smart, Arabic, mobile-first system, and it achieved that goal. The delivered system automates the full operational cycle — receiving devices, selling goods, delivering with payment, and reporting — on top of a unified balance accounting model that keeps every customer’s financial position accurate and traceable. It is built on a secure three-tier, cloud-deployed architecture with multi-tenant data isolation. Most importantly, the system was validated inside a real, operating shop, where reconciling each customer’s balance against their transactions and deposits confirmed the correctness of the accounting model. The result is a practical demonstration of digital transformation for a small business under difficult infrastructure conditions.

All five measurable objectives defined in Section 1.2.2 were fully achieved, each verified individually against its specific numeric criterion in Section 6.6.

8.2 Future Work

- Payment-channel classification: distinguishing cash, bank accounts, and mobile wallets (e.g., Jawwal Pay, PalPay) with per-channel reports.
- Offline-first operation with a local cache and background synchronization, better matching Gaza’s connectivity conditions.
- Customer notifications when a device finishes charging.
- Formal load and stress testing to establish the system’s operational limits, and mitigation of the cloud cold-start delay.
- Profit analytics and advanced reporting, and expansion into a full software-as-a-service offering for similar shops.

References

- [1] Square, Inc., "Square Point of Sale," Square, 2024. [Online]. Available: <https://squareup.com/pos>. [Accessed: Aug. 2026].
- [2] Loyverse, "Loyverse POS — Point of Sale System," Loyverse, 2024. [Online]. Available: <https://loyverse.com>. [Accessed: Aug. 2026].
- [3] Khatabook, "Khatabook — Smart Digital Ledger for Small Business Owners," Khatabook, 2026. [Online]. Available: <https://khatabook.com>. [Accessed: Aug. 2026].
- [4] Daftra, "دفتر — برنامج محاسبة وفواتير عربي للشركات والمحلات," Daftra, 2026. [Online]. Available: <https://www.daftra.com>. [Accessed: Aug. 2026].
- [5] Qayd, "دفتر الديون الرقمي — قيد," Qayd, 2026. [Online]. Available: <https://qayd.sa>. [Accessed: Aug. 2026].
- [6] W. W. Royce, "Managing the development of large software systems: concepts and techniques," in Proc. 9th Int. Conf. on Software Engineering (ICSE '87), Washington, DC, USA, 1987, pp. 328–338.
- [7] R. T. Fielding and R. N. Taylor, "Principled design of the modern web architecture," ACM Trans. Internet Technol., vol. 2, no. 2, pp. 115–150, May 2002, doi: 10.1145/514183.514185.
- [8] T. Härder and A. Reuter, "Principles of transaction-oriented database recovery," ACM Comput. Surv., vol. 15, no. 4, pp. 287–317, Dec. 1983.
- [9] C. J. Guo, W. Sun, Y. Huang, Z. H. Wang, and B. Gao, "A framework for native multi-tenancy application development and management," in Proc. 9th IEEE Int. Conf. on E-Commerce Technology and 4th IEEE Int. Conf. on Enterprise Computing, E-Commerce, and E-Services (CEC/EEE '07), Tokyo, Japan, 2007, pp. 551–558.
- [10] B. Gao, W. H. An, X. Sun, Z. H. Wang, and L. Fan, "A non-intrusive multi-tenant database software for large-scale SaaS applications," in Proc. 2011 IEEE 8th Int. Conf. on e-Business Engineering (ICEBE), 2011, pp. 157–163, doi: 10.1109/ICEBE.2011.39.
- [11] K. Toyama, "Mobile applications for informal economies," in Proc. IEEE Global Humanitarian Technology Conference (GHTC), 2009. [Online]. Available: <https://ieeexplore.ieee.org/document/5352703>.

Appendix A: Repository Links and Design Resources

The complete source code, design files, and test artifacts of the project are available at the following locations:

- Flutter mobile application (GitHub): <https://github.com/Hussenghbayen/testcharge>
- REST API — Node.js / Express (GitHub):
<https://github.com/MohammEdeyad2001/charging-api>
- Postman collection: included in the postman/ directory of the API repository above, covering every endpoint in Table (5.1) and the test scenarios in Table (6.1).
- Database schema (ERD): see Figure (4.2), Chapter 4.
- UI/UX design (Figma):
<https://www.figma.com/design/z7EdLLqHxaxUXPiHqY20FK/Untitled?node-id=0-1&t=MUQAse6INKcl6hVp-1>

Appendix B: Selected Source Code

The following code excerpts are copied verbatim from the project's GitHub repositories, as referenced in Chapter 5. Each excerpt is labeled with its source file and repository.

B.1 — Authentication middleware

File: src/middleware/auth.js (charging-api repository). Verifies the Firebase ID token on every request, resolves or just-in-time creates the corresponding owner record, and attaches it to req.owner so that owner_id is always taken from a server-verified source.

```
// src/middleware/auth.js
const pool = require('../config/db');
const admin = require('../config/firebase');

const authMiddleware = async (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || typeof authHeader !== 'string' || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ message: 'unauthorized: missing token' });
  }

  const parts = authHeader.split(' ');
  if (parts.length !== 2) {
    return res.status(401).json({ message: 'invalid token format' });
  }

  const token = parts[1];

  try {
    const decodedToken = await admin.auth().verifyIdToken(token);
    const firebaseUid = decodedToken.uid;
    const email = (decodedToken.email || '').toLowerCase();

    if (!firebaseUid || !email) {
      return res.status(401).json({ message: 'token missing required data' });
    }

    let ownerResult = await pool.query(
      'SELECT id, name, email, firebase_uid FROM owner
      WHERE firebase_uid = $1 OR email = $2 LIMIT 1',
      [firebaseUid, email]
    );

    if (ownerResult.rows.length === 0) {
      const name = (decodedToken.name || email.split('@')[0]).slice(0, 255);
      const emailSafe = email.slice(0, 255);
      try {
        const upsertQuery = `
          INSERT INTO owner (name, email, firebase_uid, password, created_at)
          VALUES ($1, $2, $3, $4, NOW())
          ON CONFLICT (email) DO UPDATE
            SET firebase_uid = COALESCE(owner.firebase_uid, EXCLUDED.firebase_uid)
          RETURNING id, name, email, firebase_uid
        `;
        const upsertRes = await pool.query(upsertQuery, [name, emailSafe, firebaseUid,
'firebase_auth']);
        ownerResult = { rows: [upsertRes.rows[0]] };
      } catch (upsertErr) {
        console.error('Owner upsert error:', upsertErr.message);
        ownerResult = await pool.query(
          'SELECT id, name, email, firebase_uid FROM owner
          WHERE firebase_uid = $1 OR email = $2 LIMIT 1',
          [firebaseUid, email]
        );
      }
    } else if (!ownerResult.rows[0].firebase_uid) {
      await pool.query('UPDATE owner SET firebase_uid = $1 WHERE id = $2', [firebaseUid,
ownerResult.rows[0].id]);
      ownerResult.rows[0].firebase_uid = firebaseUid;
    }
  }
}
```

```

    req.owner = ownerResult.rows[0];
    next();
  } catch (err) {
    console.error('Firebase Auth Error:', err.message);
    if (err.code === 'auth/id-token-expired') {
      return res.status(401).json({ message: 'token expired' });
    }
    return res.status(401).json({ message: 'invalid or expired token' });
  }
};

module.exports = authMiddleware;

```

B.2 — Deposit and debt-repayment logic

File: src/controllers/customerController.js, function payDebt (charging-api repository). Records a deposit, updates the customer's unified balance, and historically distributes the payment across outstanding transactions — all within a single ACID database transaction.

```

// src/controllers/customerController.js (excerpt)
const payDebt = async (req, res) => {
  const { id } = req.params;
  const { amount, note } = req.body;

  if (typeof amount === 'undefined' || isNaN(Number(amount)) || Number(amount) <= 0) {
    return res.status(400).json({ message: 'invalid amount' });
  }

  const client = await pool.connect();
  try {
    await client.query('BEGIN');

    const customerRes = await client.query(
      'SELECT * FROM customer WHERE id = $1 AND owner_id = $2 FOR UPDATE',
      [id, req.owner.id]
    );
    if (customerRes.rows.length === 0) {
      await client.query('ROLLBACK');
      return res.status(404).json({ message: 'customer not found' });
    }

    // Document the deposit
    await client.query(
      'INSERT INTO debt_payment (customer_id, amount, note, paid_at) VALUES ($1, $2, $3, NOW())',
      [id, amount, note || null]
    );

    // Core logic: add to the unified customer balance
    await client.query('UPDATE customer SET balance = balance + $1 WHERE id = $2', [amount, id]);

    // Historical distribution across outstanding transactions (oldest first)
    let remaining = parseFloat(amount);
    const pendingTransactions = await client.query(
      'SELECT * FROM transaction WHERE customer_id = $1 AND owner_id = $2 AND remaining_debt > 0 ORDER BY date ASC, received_at ASC FOR UPDATE',
      [id, req.owner.id]
    );
    for (const tx of pendingTransactions.rows) {
      if (remaining <= 0) break;
      const debt = parseFloat(tx.remaining_debt || 0);
      if (debt <= 0) continue;
      const pay = Math.min(remaining, debt);
      const newAmountPaid = parseFloat(tx.amount_paid || 0) + pay;
      const newStatus = newAmountPaid >= parseFloat(tx.amount_due || 0) ? 'paid' : 'partial';
      await client.query(
        'UPDATE transaction SET amount_paid = $1, payment_status = $2 WHERE id = $3',
        [newAmountPaid, newStatus, tx.id]
      );
      remaining -= pay;
    }
  }
};

```

```

    await client.query('COMMIT');
    const updated = await pool.query('SELECT * FROM customer WHERE id = $1', [id]);
    res.json({ message: 'payment recorded', customer: updated.rows[0] });
  } catch (err) {
    await client.query('ROLLBACK').catch(() => {});
    console.error('payDebt error:', err.message);
    res.status(500).json({ message: 'server error' });
  } finally {
    client.release();
  }
};

```

B.3 — Customers Cubit (state management)

File: lib/logic/customers/customers_cubit.dart (testcharge repository). Fetches customers from the API, attaching the Firebase ID token to every request, and records debt payments.

```

import 'dart:convert';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:http/http.dart' as http;
import 'package:firebase_auth/firebase_auth.dart';
import 'customers_state.dart';

class CustomersCubit extends Cubit<CustomersState> {
  CustomersCubit() : super(CustomersInitial());
  static const String _baseUrl = 'https://charging-api-tkne.onrender.com/api';

  Future<String> _getToken() async {
    return await FirebaseAuth.instance.currentUser?.getIdToken() ?? '';
  }

  Future<void> getCustomers() async {
    emit(CustomersLoading());
    try {
      final token = await _getToken();
      final response = await http.get(
        Uri.parse('${_baseUrl}/customers'),
        headers: {
          'Content-Type': 'application/json',
          'Authorization': 'Bearer $token',
        },
      );
      final data = jsonDecode(response.body);
      if (response.statusCode == 200) {
        emit(CustomersSuccess(customers: data));
      } else {
        emit(CustomersError(errorMsg: 'failed to load customers'));
      }
    } catch (e) {
      emit(CustomersError(errorMsg: e.toString()));
    }
  }

  Future<void> payDebt(String id, double amount, String note) async {
    emit(CustomersLoading());
    try {
      final token = await _getToken();
      final response = await http.put(
        Uri.parse('${_baseUrl}/customers/$id/pay'),
        headers: {
          'Content-Type': 'application/json',
          'Authorization': 'Bearer $token',
        },
        body: jsonEncode({'amount': amount, 'note': note}),
      );
      if (response.statusCode == 200) {
        emit(PayDebtSuccess());
        await getCustomers();
      } else {
        emit(CustomersError(errorMsg: 'failed to record payment'));
      }
    } catch (e) {
      emit(CustomersError(errorMsg: e.toString()));
    }
  }
}

```

Appendix C: User Manual (Brief)

This brief manual summarizes how a shop owner uses the application:

1. Sign in with your email and password; you will see only your own shop's data.
2. From the home screen, tap "New Device" to register a charging device: choose the item (price appears automatically), enter the shelf number, and choose "not paid" or "paid in full."
3. Tap "Quick Sale" to sell a retail item by quantity, on account or paid.
4. In "Active Devices," tap "Deliver" to hand over a device; the customer's balance is shown and you may collect a full or partial payment.
5. Open a customer to view their statement, and use "Deposit" to record a payment.
6. Use the "Debts" screen to see who owes money, and the "Log" screen for daily, weekly, or monthly reports.